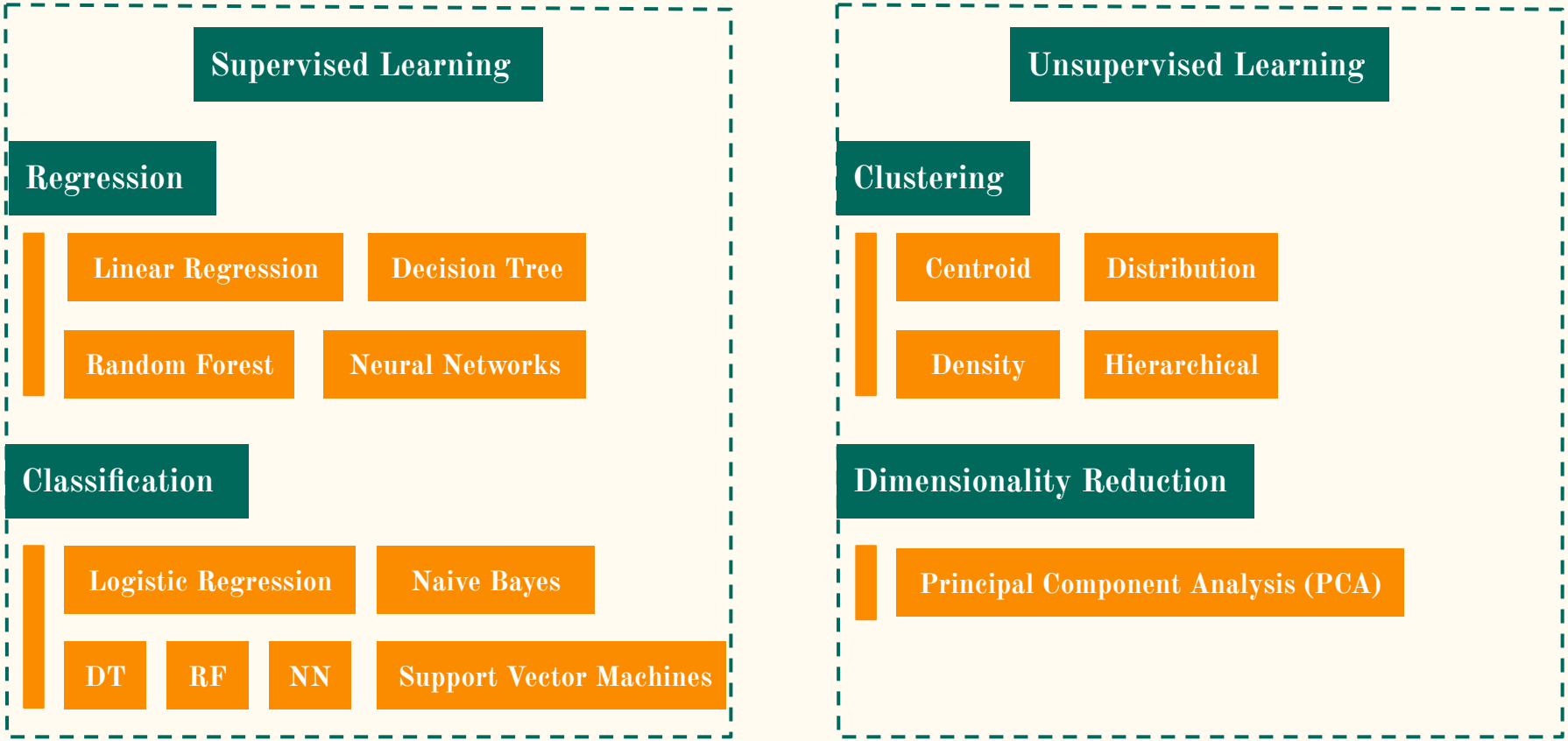
Machine Learning Workshop | L02

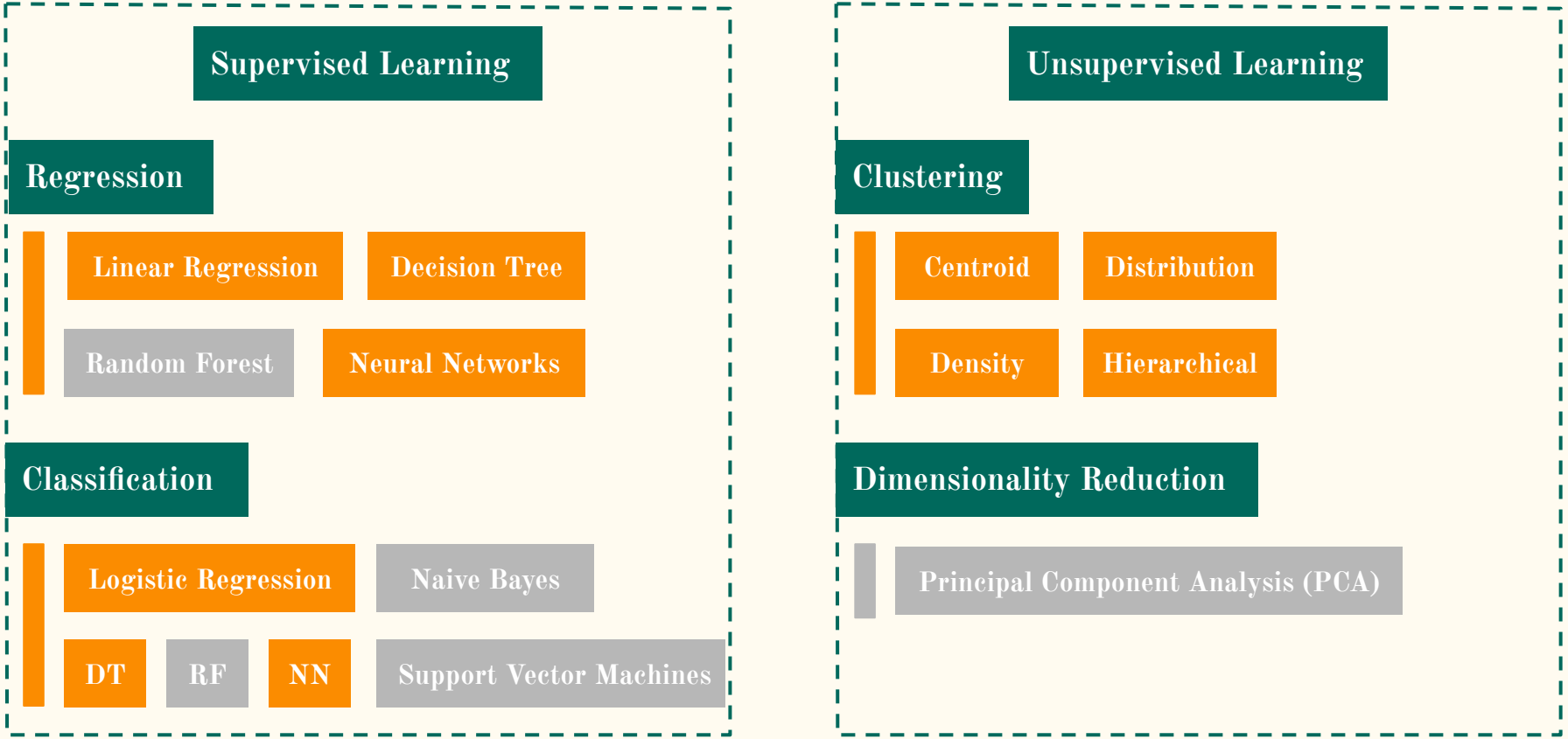
—

Unsupervised Learning | Deep Learning

Quick recap : the big picture of ML



This workshop will cover :



Outline

Regression

- Concept of regression
- Linear, non-linear regression
- Practical demonstration

Unsupervised Learning : Clustering

- Motivations : Why do we adopt unsupervised learning ?
- Intro to clustering (sklearn framework)
- Overview of clustering algorithms (sklearn.cluster)
- Practical demonstration (K-mean, Mean-shift, (H) DBSCAN)
- Validation by silhouette scores (sklearn.metrics)

Introduction to Deep Learning 1

- Neural networks, computer vision, history and paradigm shift of Deep Learning
- Hardware knowledge (GPU, CPU, cores)

We are now :

Supervised Learning

Regression

Linear Regression

Classification

Unsupervised Learning

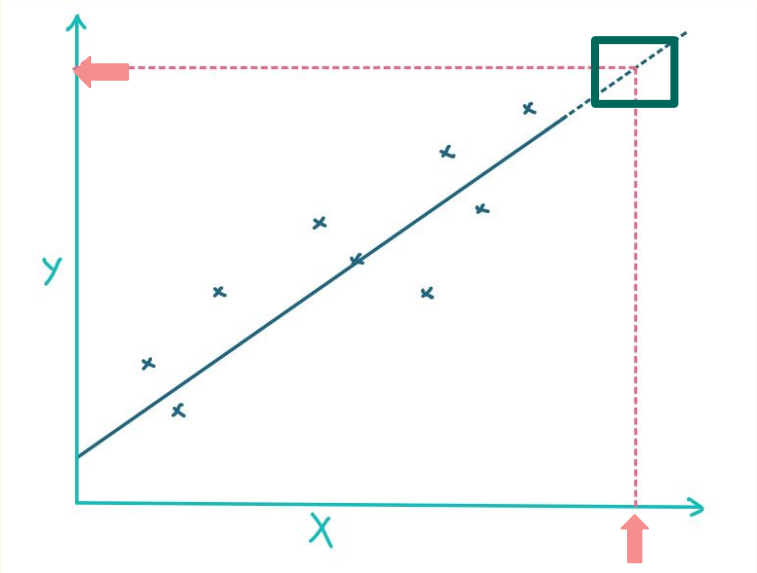
Clustering

Dimensionality Reduction

Motivation of regression

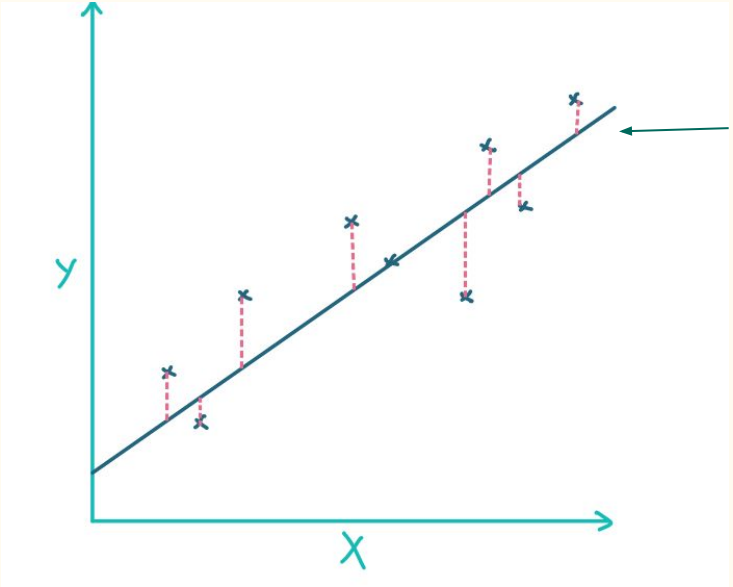
Motivation : Discover the unknown **relationship** between data

Aim » make predictions with the **trend**



Concept of regression

(Linear) Regression : Find a line to fit the data



Least sqaure method

Line of Best-fit

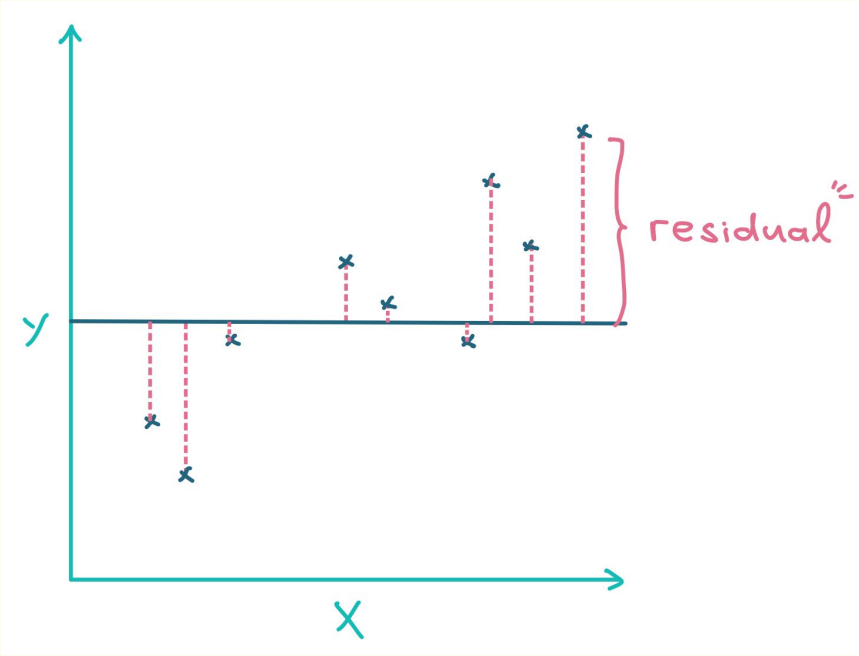
- X : independent variables
- y : dependent variable

Linear regression : Least-sqaured method

(1) Add a random straight line
(2) Calculate **residuals**
(3) Calculate **sum of squared residuals**

Key concept | **Residual / Bias / Error term**
the distance from each data point to the line

Key concept | **Sum of squared residuals**
square each residual and sum up



Mean square error (MSE), Residuals, and error term

Residual(=MSE) and error term are 2 concepts often being interchanged, but there is a formal difference.

error term : observed data vs actual population (unobservable)

residuals : observed data vs sample population (observable)

Error Term / Bias includes :

(1) non-linearity, (2) unpredictable effects,

(3) ommitted variables and (4) measurement errors

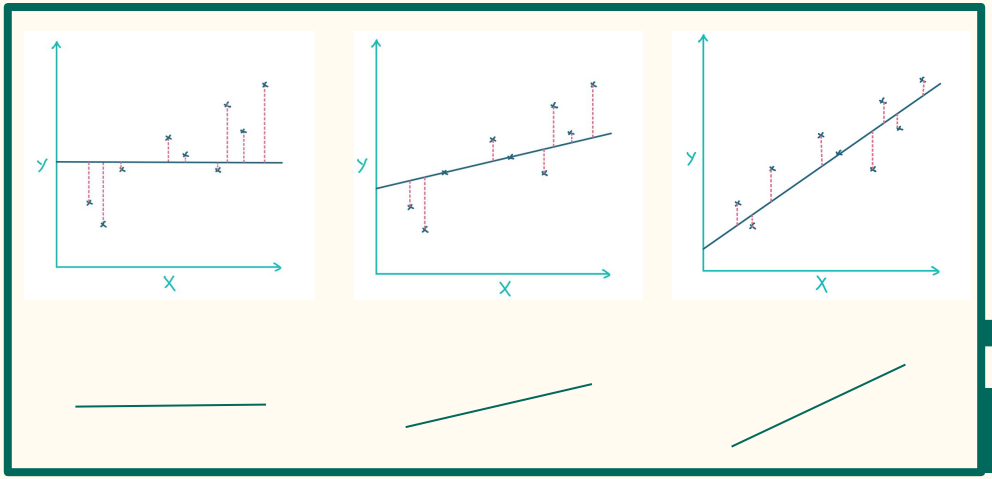
But we will treat them as interchangeable and equivalent concepts in this workshop

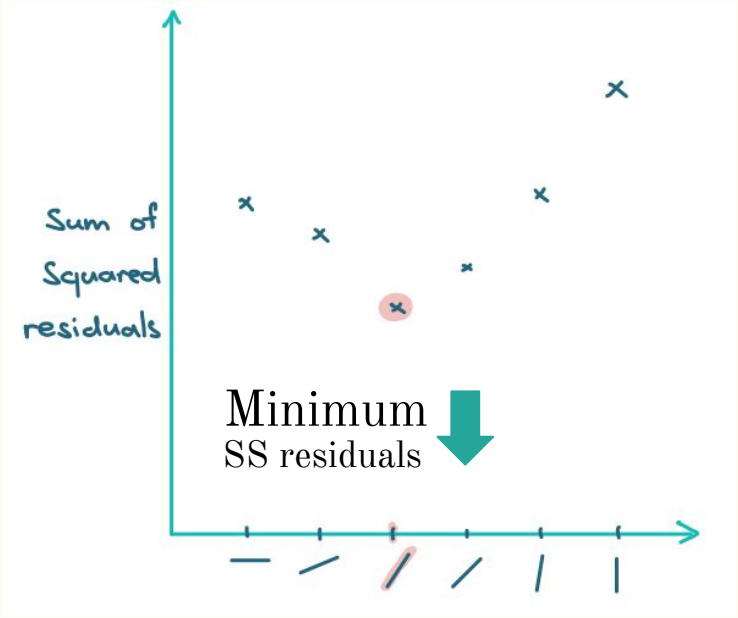
Least-squared method (cont)

(4) Rotate the straight line

(5) Minimize the sum of squared residuals

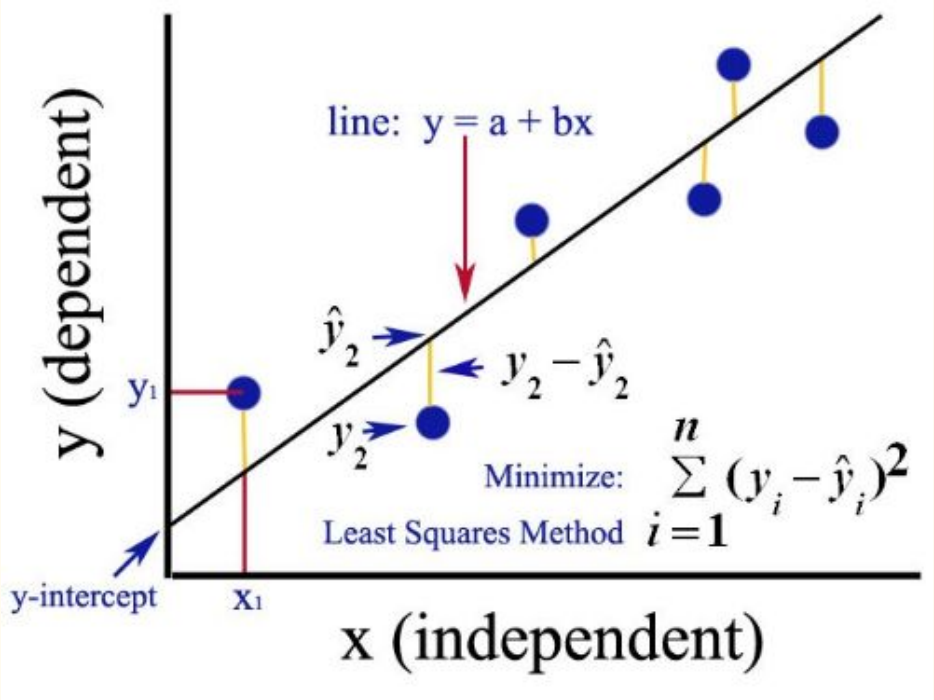
>> Line of best-fit





Take SS residuals

Least-squared method (statistical form)



Ordinary Least square and gradient descent

Least sqaure method

- statistical method

  minimize the sum of sqaured residuals
- use in bivariate model (1 X → 1 y)
- can be used in multivariate model (n X → 1 y)

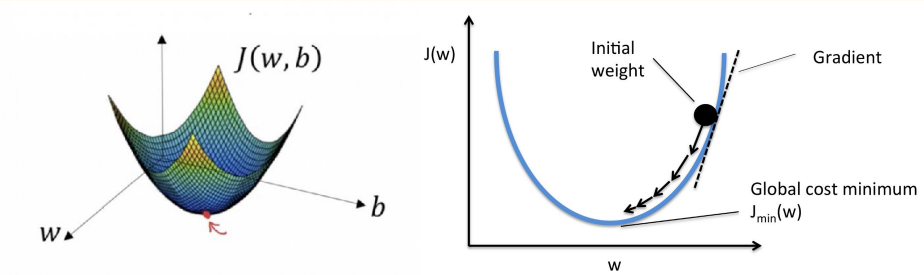
  (but computationally expensive)

Gradient Descent

- within calculus family

  down the hill at steepest gardient direction

  until reaching the valley (local minimum)



Back to this concept in depth later

Assumptions made in linear regression model

1. Linear and Additive
2. Independent variables should not be correlated
3. No correlation between the residual (error) terms
4. Error term must have constant variance (homoskedasticity)
5. Error term must be normally distributed

What if ... the assumptions are violated ?

1. non-linear / non-additive → inefficient model
2. residuals are dependent from each other → autocorrelation
3. independent variables are highly correlated → multicollinearity
4. non-constant variance in error term → heteroskedasticity
5. non-normally of error terms → Confidence interval gets too wide / narrow

Univariate Linear regression with Scikit-learn

Dependencies / Package
sklearn > linear_model > LinearRegression
sklearn.metrics > mean_squared_error, r2_score

```
# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(X, y)

# Make predictions using the testing set
diabetes_y_pred = regr.predict(X)
```

3 key steps

1. Create object (regression model)
2. Train model.fit(X, y)
3. Test and predict model.predict(X)

Univariate Linear Regression : coefficients

$y = ax + b$

	geometrical meaning	scikit-learn parameters (model = regression object)	return
a	slope	coefficient model.coef_	1D list (1 independent variables → 1 elements)
b	vertical intercept	intercept model.intercept_	float

Example 1 : diabetes dataset from sklearn

1. Pre-processing

```
# Load the diabetes dataset
diabetes_X, diabetes_y = datasets.load_diabetes(return_X_y=True)

# Use only one feature
diabetes_X = diabetes_X[:, np.newaxis, 2]

# Split the data into training/testing sets
diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]

# Split the targets into training/testing sets
diabetes_y_train = diabetes_y[:-20]
diabetes_y_test = diabetes_y[-20:]
```

Example 1 : diabetes dataset from sklearn

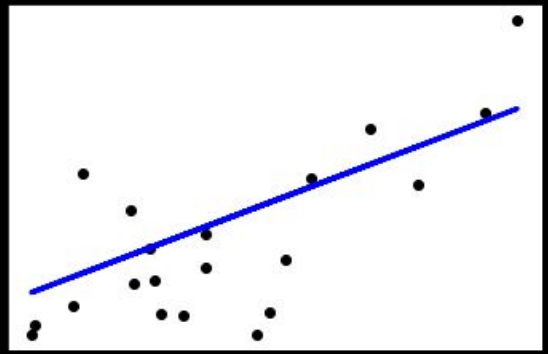
2. Regression

```
# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(diabetes_X_train, diabetes_y_train)

# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X_test)
```

Example 1 : diabetes dataset from sklearn

3. Plot results

```
# Plot outputs
plt.scatter(diabetes_X_test, diabetes_y_test, color="black")
plt.plot(diabetes_X_test, diabetes_y_pred, color="blue", linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



Example 1 : diabetes dataset from sklearn

4. Validation

```
# The coefficients (slope and intercept)
print("Model slope: ", regr.coef_[0])
print('Model intercept: ', regr.intercept_)

# The mean squared error ()
print("Mean squared error: %.2f" % mean_squared_error(diabetes_y_test, diabetes_y_pred))

# The coefficient of determination: 1 is perfect prediction
print("Coefficient of determination: %.2f" % r2_score(diabetes_y_test, diabetes_y_pred))
```

Validation of regression

1. Mean Absolute Error (MAE) - robust to outliers
2. Mean Squared Error (MSE) - susceptible to outliers
3. Root Mean Squared Error (RMSE) - susceptible to outliers

---

4. R-Squared (Coefficient of Determination)
5. Adjusted R-Squared
6. Standard Error

Validation of regression : Coefficient of determination

[Coefficient of determination / R^2 : **- Goodness of fit -**]

R^2 : the proportion that the **variation in y** can be explained by the regression of X

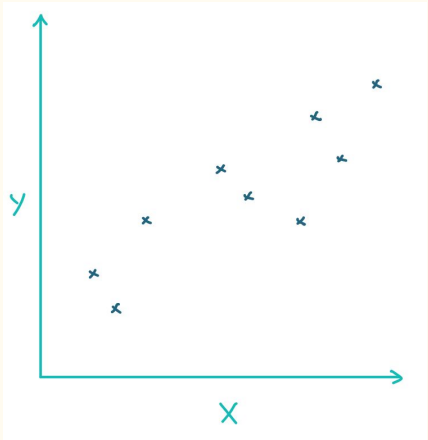
$$R^2 = \frac{\text{Variance explained by the model}}{\text{Total variance}}$$

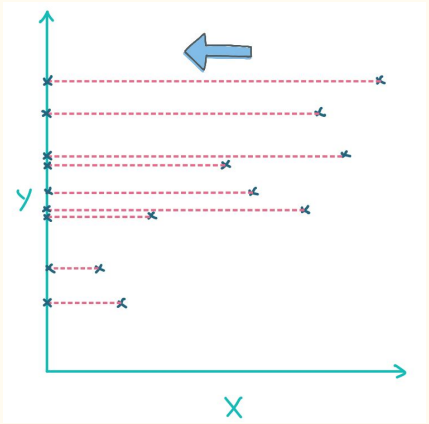
(R^2 varies from 0 to 1 | higher is better)

Motivation :

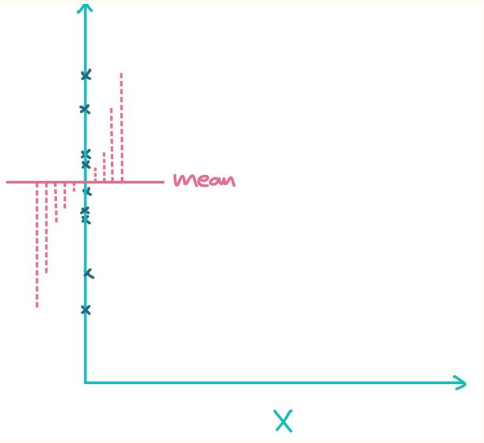
When we predict unkown y with given X value, we want to know how well the regression / the regression line can explain how many percent of the regression.

Calculation (1) : Var(mean)





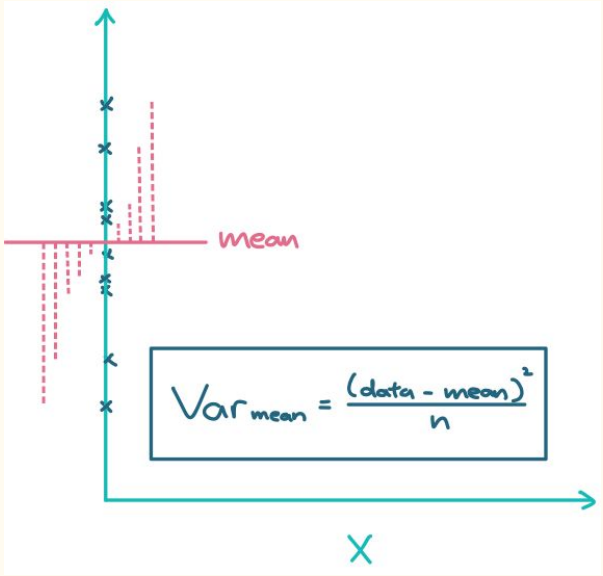
Only consider the dependent variable y of the dataset
(Neglect X value)

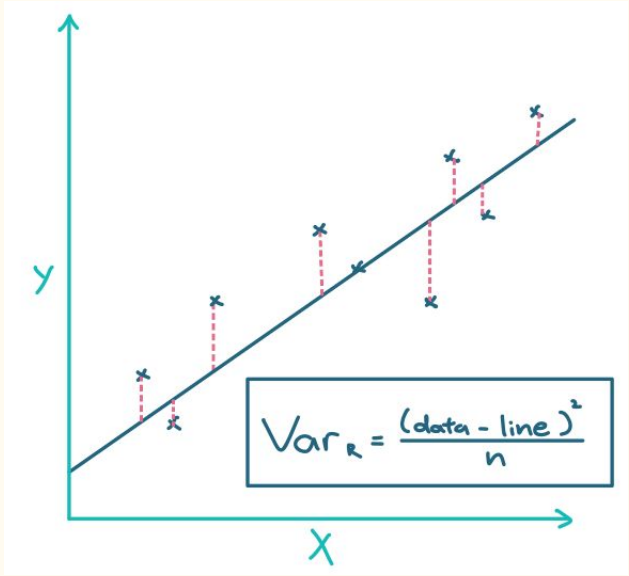


Core steps !

(1) Compute the mean of y
(2) Find the variance around the mean

Calculation (2) : R^2





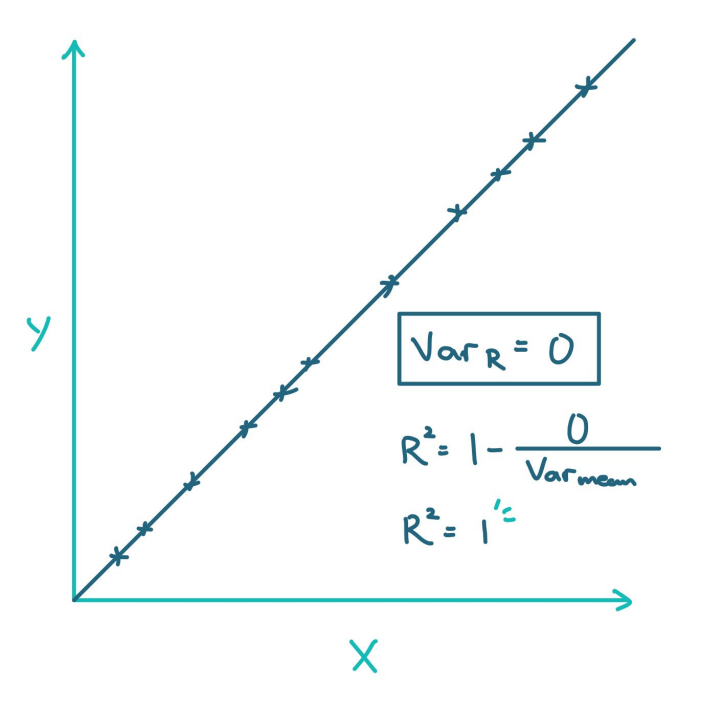
$$R^2 = \frac{Var_{mean} - Var_R}{Var_{mean}}$$

$$= 1 - \frac{Var_R}{Var_{mean}}$$

R^2 : 0 ~ 1

If Var(mean) > Var(R) :
R^2 < 0

Extended example : perfect fit



Validation of regression : standard error

- Standard error : measure the accuracy of regression
- Recall that regression line is formed by minimizing the sum of sqaured residuals
- The standard error is calculated with the mean sqaured error by the following formula : (seems like standard deviation)

$$\sigma = \sqrt{\frac{\Sigma(Y - \hat{Y})^2}{N-2}}$$

Y : actual value (dependent variable)

$\hat{Y}$: prediction (dependent variable)

$Y - \hat{Y}$: error

$\Sigma(Y - \hat{Y})$: sum of squared error.

Extension : why N-2 ?

Recall sample SD, we put N-1 instead of N in the denominator.

Reason behind : population mean is unknown and sample mean is used to estimate. The data will be closer to sample mean that it will be on the population mean. Compensation is made by dividing N-1 instead of N. (Bessel's correction)

Why N-1 ?

If we know the sample mean, and we know all the values of data except the remaining one, we can indeed calculate what the remaining value must be.

(n-1 degrees of freedom)

Extension : why N-2 ?

Recall the linear regression, we put N-2 instead of N in the denominator.

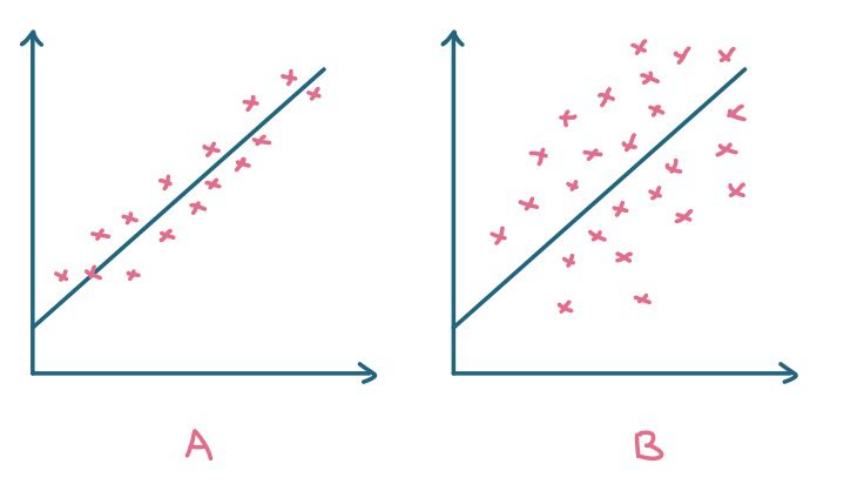
Reason behind : Compensate with a larger denominator (Bessel's correction)

Why N-2 ?

This time we know (1) intercept, and (2) slope of the regression line, assume we have the standard error and all values except the two remaining values of data, we can calculate what they must be. (n-2 degrees of freedom)

Concept of standard error

The smaller the standard error, the more accurate the linear regression is.



Which case has a smaller standard error ?

$A < B$

Multivariate Linear Regression

We can extend the linear regression with more dimensions.

E.g. two-dimensional data

$$\hat{y}(w, x) = w_0 + w_1 x_1 + w_2 x_2$$

	component
Intercept	w_0
coefficients	w_1 w_2

One-dimensial data

$$\hat{y}(w, x) = w_0 + w_1 x_1$$

	component
Intercept	w_0
coefficients	w_1

more than 1 independent variables X

Statistics formulation of multiple regression

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^{\mathsf{T}} \boldsymbol{\beta} + \varepsilon_i, \qquad i = 1, \ldots, n,$$

where $^{\mathsf{T}}$ denotes the transpose, so that $\mathbf{x}_i^{\mathsf{T}}\boldsymbol{\beta}$ is the inner product between vectors $\mathbf{x}_i$ and $\boldsymbol{\beta}$.

$$\{y_i,\, x_{i1}, \ldots, x_{ip}\}_{i=1}^{n}$$

Often these *n* equations are stacked together and written in matrix notation as

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon},$$

where

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad X = \begin{pmatrix} \mathbf{x}_1^{\mathsf{T}} \\ \mathbf{x}_2^{\mathsf{T}} \\ \vdots \\ \mathbf{x}_n^{\mathsf{T}} \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix}, \quad \boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

Multivariate Linear Regression : coefficients

$$\hat{y}(w, z) = w_0 + w_1 z_1 + w_2 z_2 + w_3 z_3 + w_4 z_4 + w_5 z_5$$

	scikit-learn parameters (model = regression object)	return
w_i	coefficient model.coef_	1D list (n independent variables → n elements)
w_0	intercept model.intercept_	float

Example 2 : housing price in oregon dataset from github

1. Load dataset from github

```
df =
pd.read_csv('https://raw.githubusercontent.com/satishgunjal/datasets/master/multivariate_housing_prices_
in_portlans_oregon.csv')
print('Dimension of dataset= ', df.shape)

# To get first n rows from the dataset
# default value of n is 5
df.head()
```

Example 2 : housing price in oregon dataset from github

2. pre-processing of training dataset

```
# get input values from first two columns
X = df.values[:, 0:2]

# get output values from last coulmn
y = df.values[:, 2]

# Number of training examples
m = len(y)
n = X.shape[1]

print('Total no of training examples (m) = %s \n' %(m))
print('Number of input feature (n) = %s \n' %(n))

# Show only first 5 records
for i in range(5):
   print('X =', X[i, ], ', y =', y[i])
```

Example 2 : housing price in oregon dataset from github

3. Regression (normalize within regression object)

```
# Create object of linear regression [enable normalization]
model_ols =  linear_model.LinearRegression(normalize=True)

# Note the first parameter(feature) is must be 2D array(feature matrix)
model_ols.fit(X,y)
```

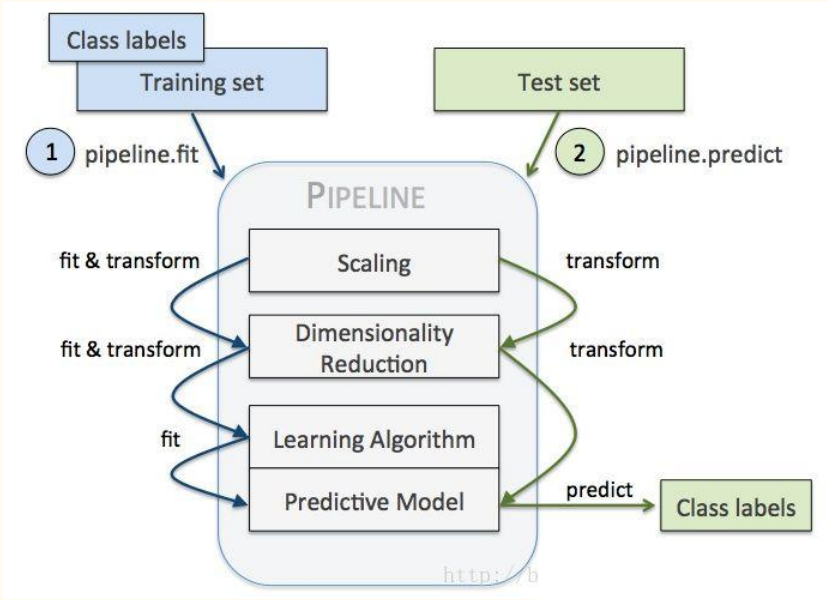
Example 2 : housing price in oregon dataset from github

4. Coefficient and intercept

```
coef = model_ols.coef_
intercept = model_ols.intercept_
print('coef = ', coef)
print('intercept = ', intercept)
```

Streaming workflow with pipelines [sklearn]

When we have multiple steps to perform on the dataset, we can packed it into a pipeline to streamline the workflow.



Eg. StandardScalar → PCA → Logistic Regression

Middle step : transformer

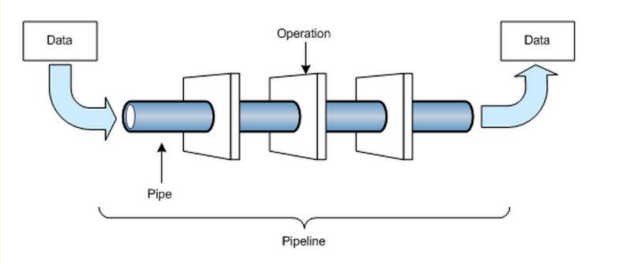
1. StandardScalar
2. PCA

fit_transform

Last step : estimator

1. Logistic Regression

fit



Example 2 : alternative modelling with pipeline

3. Regression (pipeline : normalize → linear regression)

```
# Streaming workflow with pipelines
model_ols = make_pipeline(StandardScaler(with_mean=False), linear_model.LinearRegression())
model_ols.fit(X,y)
```

Example 2 : alternative modelling with pipeline

4. Coefficient and intercept

```
coef = model_ols.named_steps['linearregression'].coef_
intercept = model_ols.named_steps['linearregression'].intercept_
print('coef = ', coef)
print('intercept = ', intercept)
```

Example 2 : alternative modelling with pipeline

5. Validation and predict

```
plt.scatter(y, model_ols.predict(X))
plt.xlabel('Price From Dataset')
plt.ylabel('Price Predicted By Model')
plt.rcParams["figure.figsize"] = (10,6) # Custom figure size in inches
plt.title("Price From Dataset Vs Price Predicted By Model")

price = model_ols.predict([[2104,    3]])
print('Predicted price of a 1650 sq-ft, 3 br house:', price)
```

Recall : validation of regression : R-sqaured

R-sqaured measure the goodness-of-fit

(proportion of data which can be explained by the regression model)

$$R^2 = \frac{Var_{mean} - Var_R}{Var_{mean}}$$

$$= 1 - \frac{Var_R}{Var_{mean}}$$

Validation of regression : problem of R-sqaured

Add useful independent variable X

↓

Residual sum of sqaures decreases

↓

R-sqaured increases

worst case

Add useless independent variable X

↓

Residual sum of sqaures doesn't change

↓

R-sqaured doesn't change

Biased estimator

R-sqaured **never decreases** → may not reflect goodness-of-fit

$$\uparrow R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \downarrow$$

Problems of high R-squared value

(1) Biased estimator (R-sqaured increases when u add more independent r.v.)

(2) Overfitting (Low Bias but High Variance)

(3) Time-series data (both X and y include trend over time)

(4) self-defined variables (X: income → y: poverty rate)

Introducing adjusted R-sqaure to penalize the inflation of R-sqaured

Adjusted R-sqaured

- Penalize the inflation of R-sqaured

$$R^2_{adj} = 1 - \left[\frac{(1-R^2)(n-1)}{n-k-1}\right]$$

$$R^2_{adj} = 1 - \left(\frac{n-1}{n-k-1}\right)(1-R^2)$$

n : number of samples
k : number of independent variables

Adjusted R-sqaured : How it works ?

Add useless independent variable X

k increases | R^2 no change

(n - k - 1) decreases

↓

(n-1) / (n-k-1) increases

↓

adjusted R-sqaured decreases

Conclusion :

1. Adding useless independent variables will decreases the adjusted R-squared

2. Adding related independent variables will

   (1) cause a significant boost in R^2,
   (2) compensating the penalty applied with k,
   (3) increasing the adjusted R-sqaured

As k increases :

$$R^2_{adj} = 1 - \left(\frac{n-1}{n-k-1}\right)(1-R^2)$$

Range of value of adjusted R-sqaured

When $k = 0$:

$$R^2_{adj} = 1 - \left(\frac{n-1}{n-0-1}\right)(1-R^2) = 1 - (1-R^2) = R^2$$

$\therefore$ for $k > 0$, $R^2_{adj} < R^2$

Polynomial Regression : Recap on Polynomial

$$y = ß_0 + ß_1x + ß_2x^2 + \ldots + ß_nx^n$$

- y is the response variable we want to predict,
- x is the feature,
- $ß_0$ is the y intercept,
- the other $ß$s are the coefficients/parameters we'd like to find when we train our model on the available x and y values,
- n is the degree of the polynomial (the higher n is, the more complex curved lines you can create).

Linear and polynomial regression

We can extend the linear regression with polynomial features from coefficients.

E.g. degree n

$$y = ß_0 + ß_1x + ß_2x^2 + \dots + ß_nx^n$$

	component
Intercept	$ß_0$
coefficients	ßs

degree 1

$$y = ß_0 + ß_1x + ß_2x + \dots + ß_nx$$

	component
Intercept	$ß_0$
coefficients	ßs

Which returns back to basic linear regression !

So linear regression is just one of the polynomial regression with degree 1

Polynomial regression with scikit-learn

sklearn.preprocessing.PolynomialFeatures()

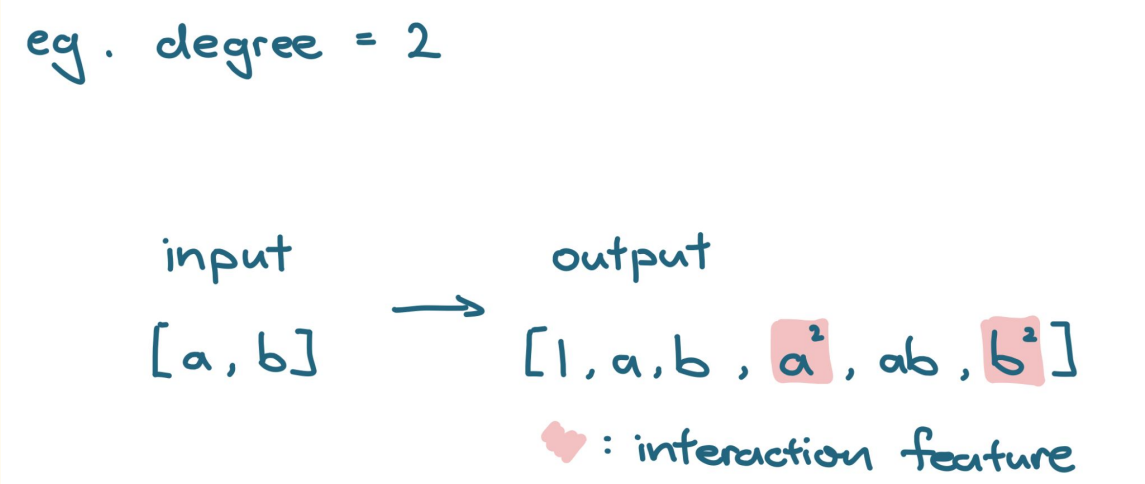
→ expand features into polynomial features

Parameters	**degree** [int / tuple] | default = 2 int : max degree tuple : (min, max)
	interaction_only [boolean] | default = False True : omit terms consist of single feature, leaving interaction only
	include_bias [boolean] | default = True True : include intercept terms (degree 0)

Polynomial regression with scikit-learn

sklearn.preprocessing.PolynomialFeatures()

→ generate polynomial feature with interaction



Example 3 : sin function

0. Generate dataset

```
def true_fun(X):
    return np.cos(1.5 * np.pi * X)


np.random.seed(0)

n_samples = 30
# for demonstraing overfit/underfit
degrees = [1, 4, 15]

X = np.random.rand(n_samples)
y = true_fun(X) + np.random.randn(n_samples) * 0.1
plt.plot(X,y,'o')

plt.figure(figsize=(8, 5))
```

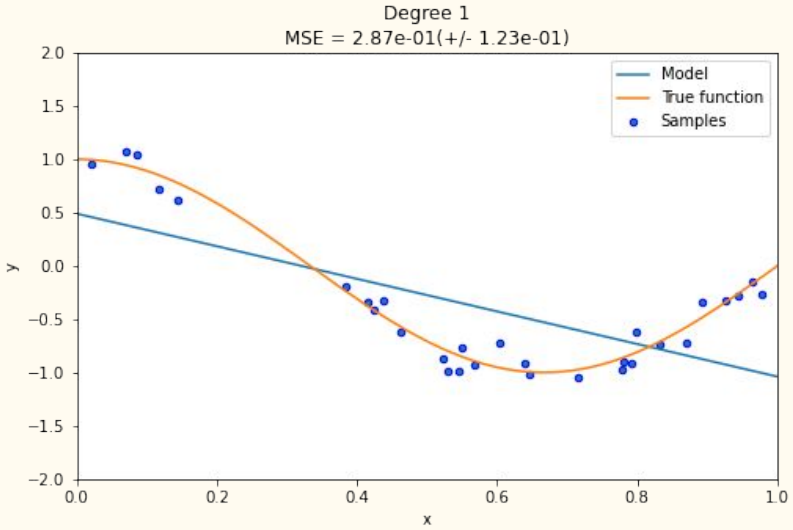
Example 3 : sin function

1. Generate dataset

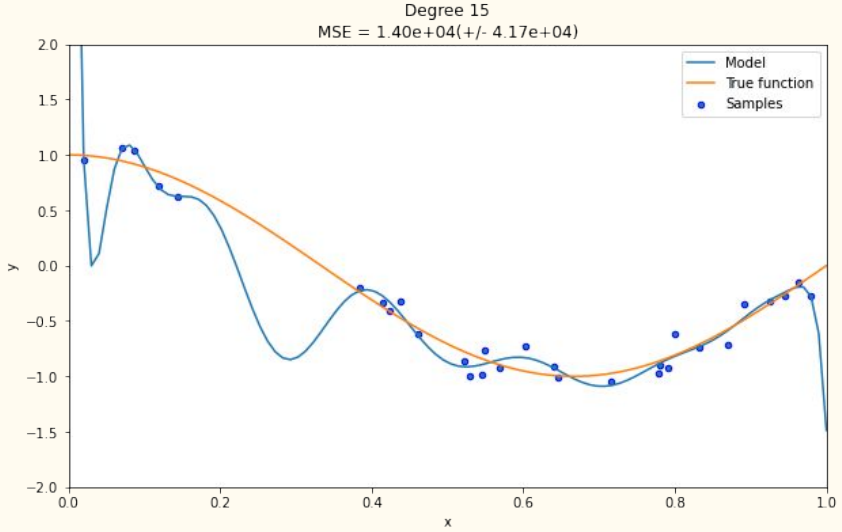
```
def true_fun(X):
    return np.cos(1.5 * np.pi * X)


np.random.seed(0)

n_samples = 30
# for demonstraing overfit/underfit
degrees = [1, 4, 15]

X = np.random.rand(n_samples)
y = true_fun(X) + np.random.randn(n_samples) * 0.1
plt.plot(X,y,'o')

plt.figure(figsize=(8, 5))
```

Underfitting and Overfitting

Underfitting



Overfitting



How to we determine underfitting or overfitting ? - Bias and variance

Bias and variance

Bias

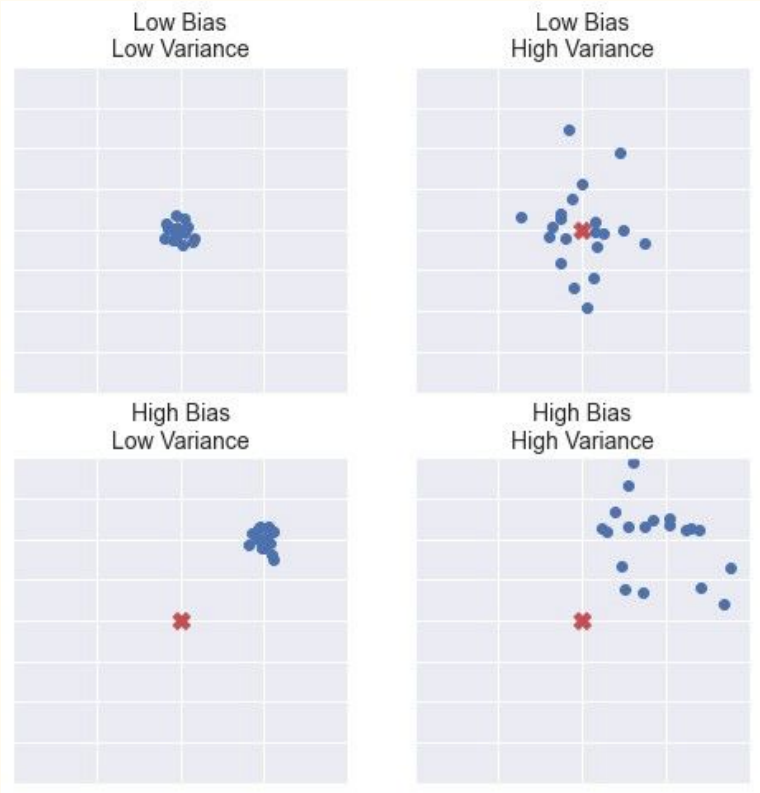
Assumption made by the model

Error of the training data

Variance

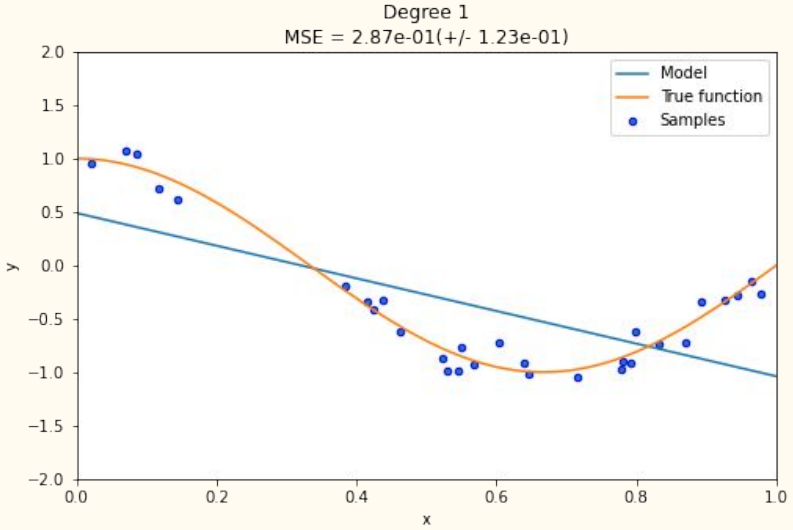
Difference between the error rate of training data and the error rate of testing data

How generalized the model is



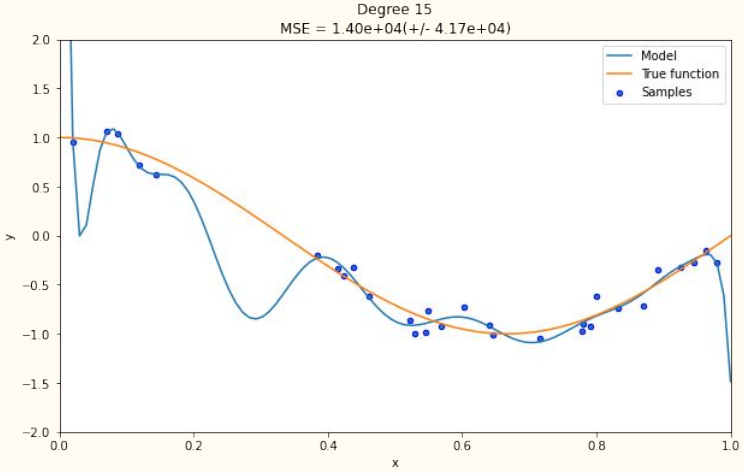
Underfitting

- Model is too simple for data
- High bias & Low variance
- Model can do accurate predictions but the initial assumption is wrong

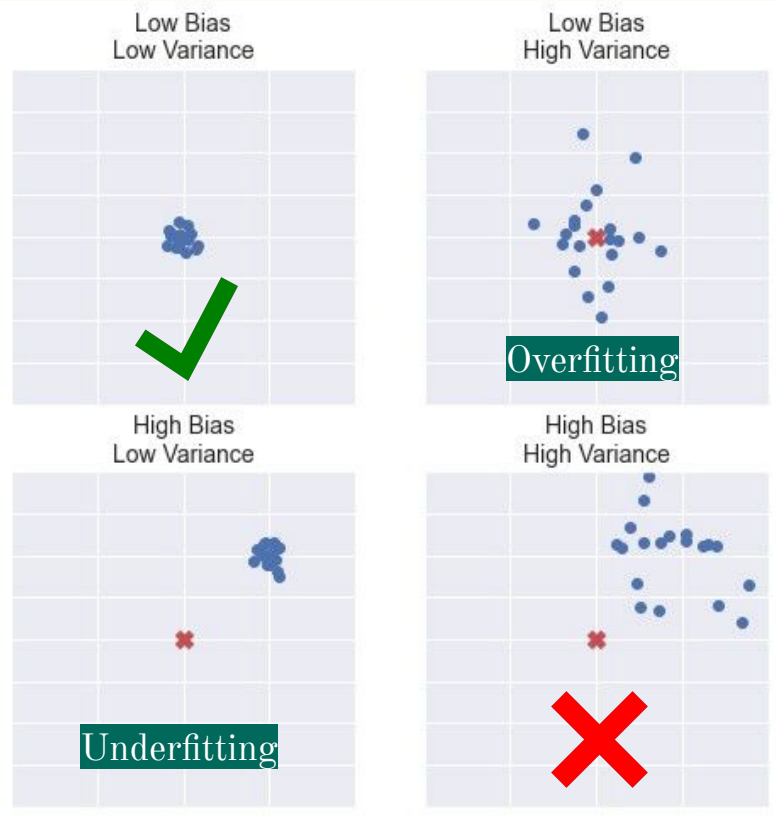


Overfitting

- Model is too complex for data
- Low bias & High variance
- Model cannot do accurate predictions
- Model is too sensitive to the dataset, when the input data is changed a bit, the output changed a lot (not generalized)



Underfitting and overfitting



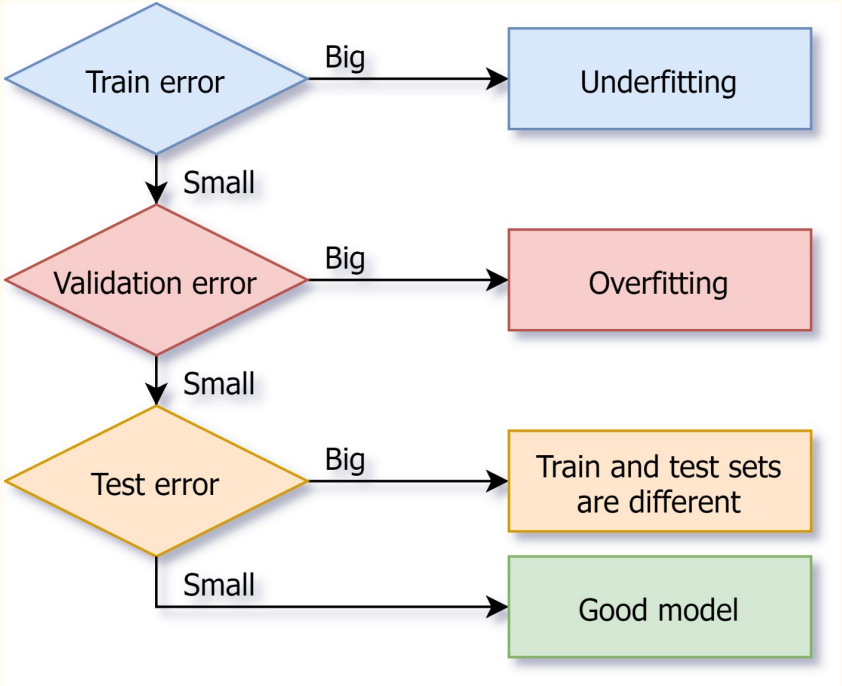
Overfitting : Low Bias | High Variance

The model outputs very different predictions from similar data

Underfitting : High Bias | Low Variance

The model outputs similar predictions from similar data, but wrong predictions (algorithm ‘miss’)

How to diagnose overfitting or underfitting ?



Quick summary : Linear regression

Linear regression : Fit a line to relate weight(X) and height(y)

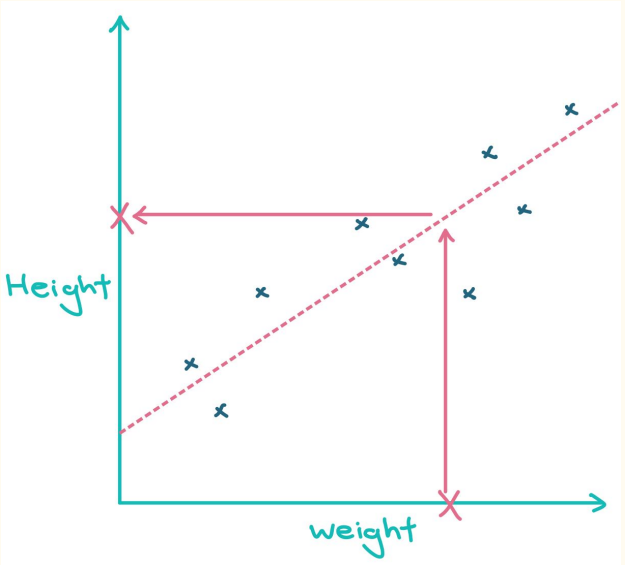
Method : Least-sqaured method (OLS) / Gradient descend

→ Use the line to predict height from the given weight

→ Using data to predict something : machine learning

Multiple regression : use weight and foot size to predict height

Polynomial regression : fit curves to describe the relationship



We are now :

Supervised Learning

Regression

- Linear Regression

Classification

- Logistic Regression

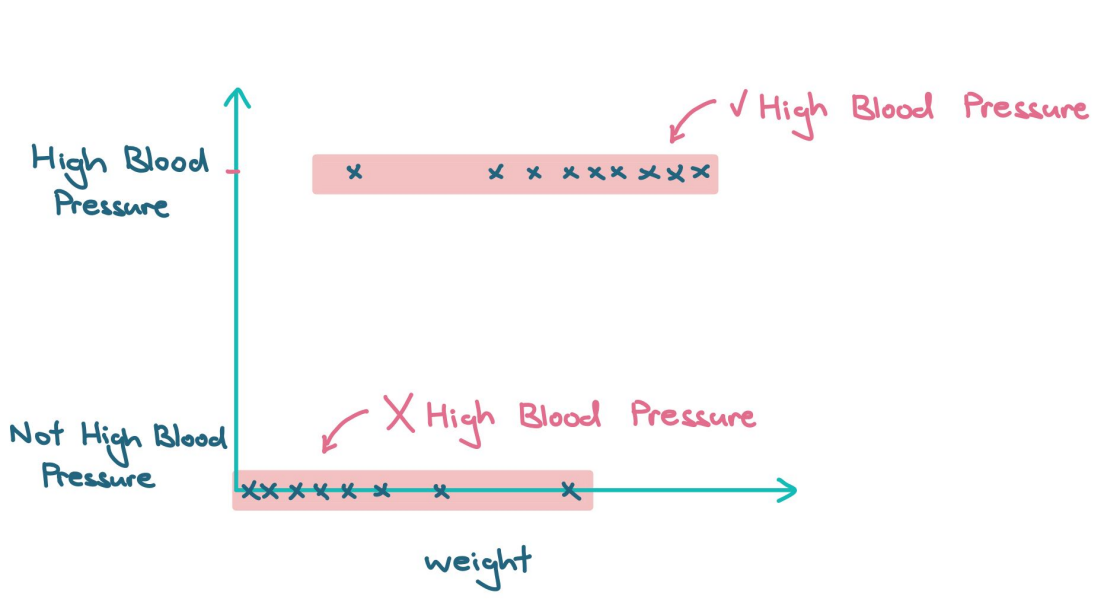
Unsupervised Learning

Clustering

Dimensionality Reduction

Logistic Regression for classification

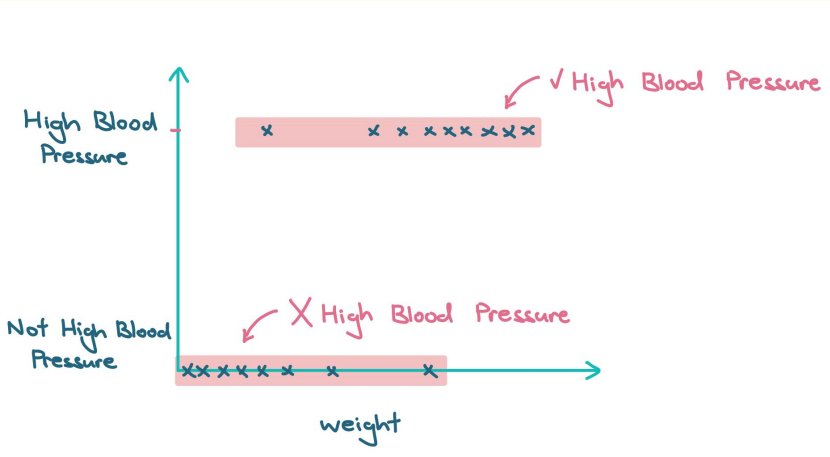
Assume we have a dataset on the health condition of people in US, and we want to predict if the person suffers from high blood pressure or not based on his/her weight



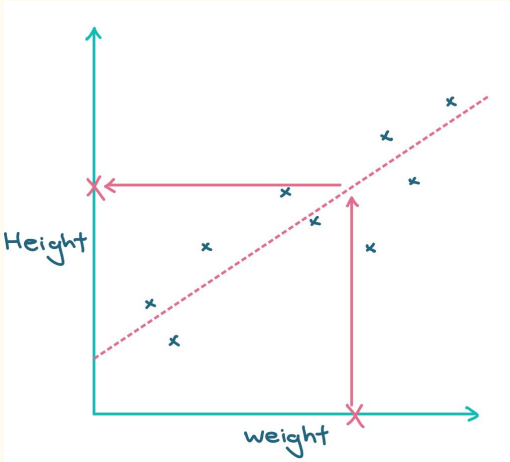
Linear regression VS Logistic regression

Logistic regression is similar to linear regression , except...

Logistic regression predicts whether something is **True or False**

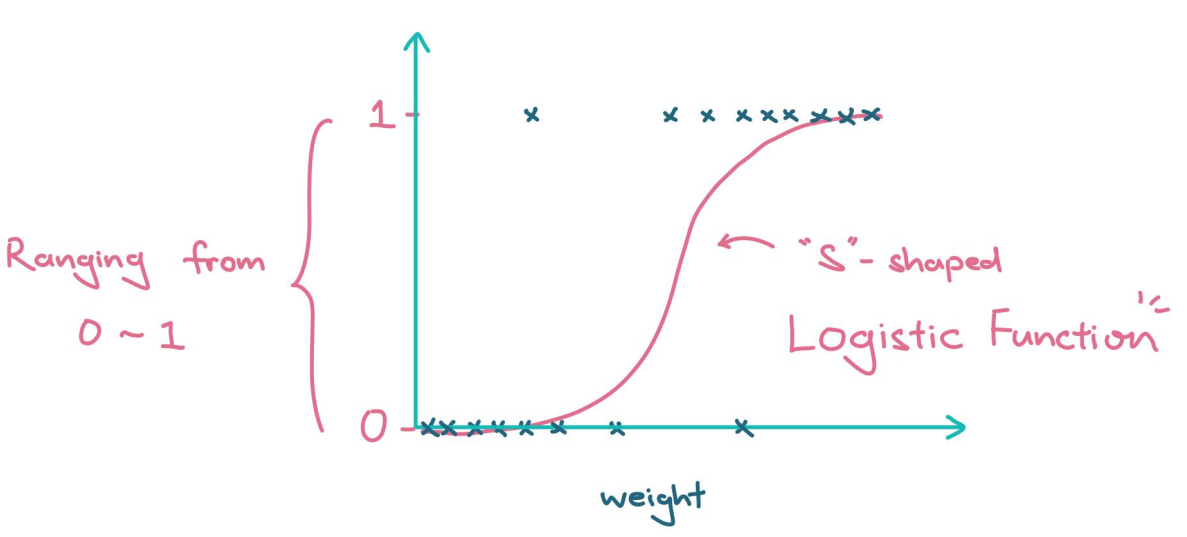


Linear regression predicts something **Continuous** (e.g. weight, height)

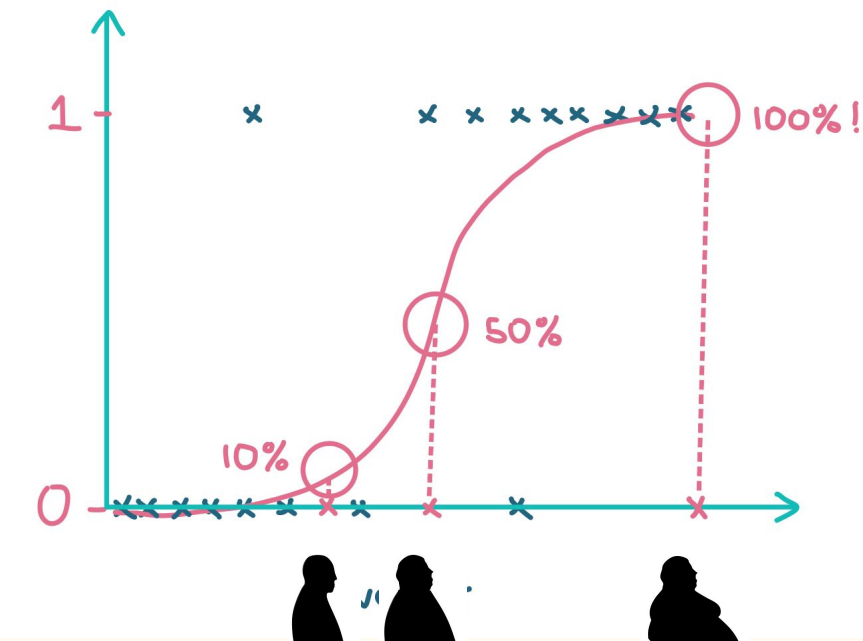


Linear regression VS Logistic regression (cont)

- Instead of fitting a line, we fit an ‘S’-shaped logistic curve
- Logistic curve ranges from 0 to 1, revealing the probability of high blood pressure based on weight (that is why it is used for classification)



Logistic Regression for classification (cont)



For example:

Prob $>$ 50 % - classify as high blood pressure

Prob $<$ 50 % - classify as not high blood pressure

Motivation : why logistic regression ?

1. Logistic regression introduces a **decision threshold** – the threshold value holds significance because it directly affects the **classification problem** and regression model. This threshold allows the value to range strictly between 0 and 1.
2. Logistic Regression is used when the **dependent variable is categorical**.

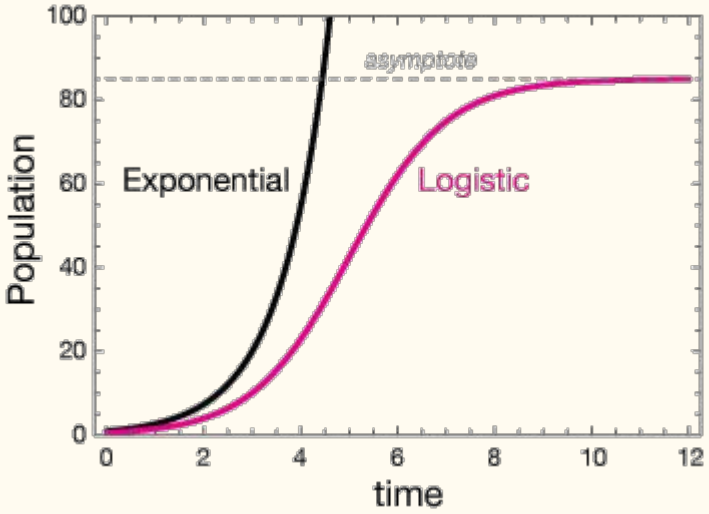
   [Linear or logistic ?]

3. A linear model cannot be used for classification problems because it is unbounded, meaning that no classification-based threshold can be made. Therefore, we use *logistic regression*.
4. Linear regression assumes that the data follows a linear function, while logistic regression models the data using the sigmoid function.

Logistic function

Exponential function : unlimited growth of population

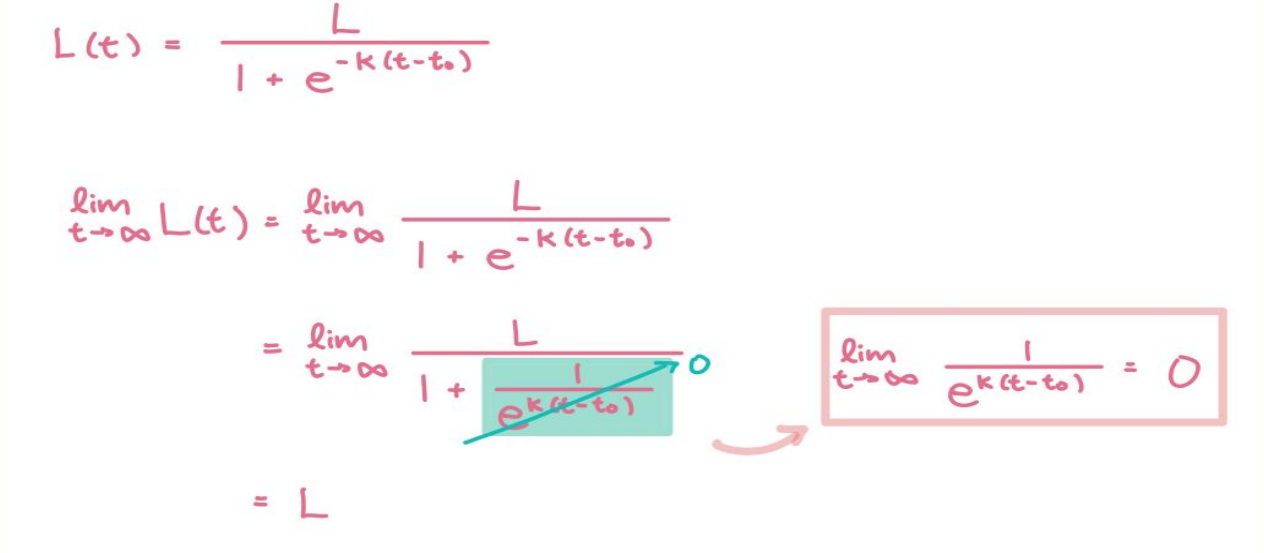
Logistic function : exponential growth with limiting factor, reaching the ceiling eventually



$$L(t) = \frac{L}{1 + e^{-k(t-t_0)}}$$

$$E(t) = ae^{kt}$$

Logistic function : horizontal asymptote L



Logistic differential equation and Logistic equation

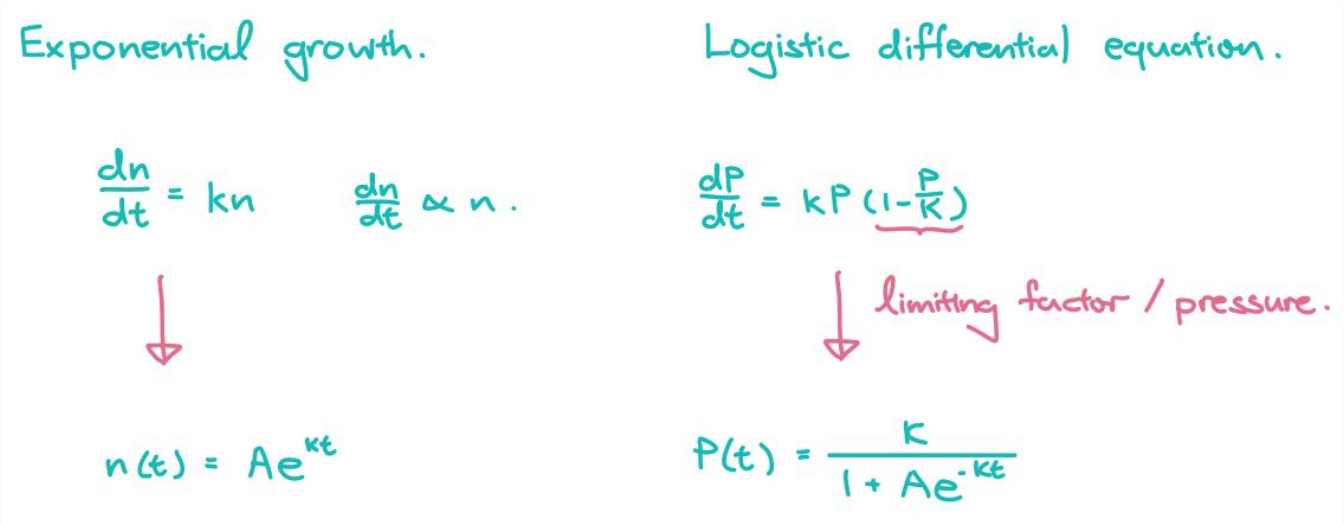
Logistic differential equation

$$\frac{dn}{dt} = kn\,(L - n)$$

Logistic function

$$n(t) = \frac{L}{1 + e^{-k(t-t_0)}}$$

Logistic differential equation and Logistic equation



Transformation of Logistic function to sigmoid function

Since logistic regression requires a curve ranging 0 - 1, we will set the asymptote L = 1

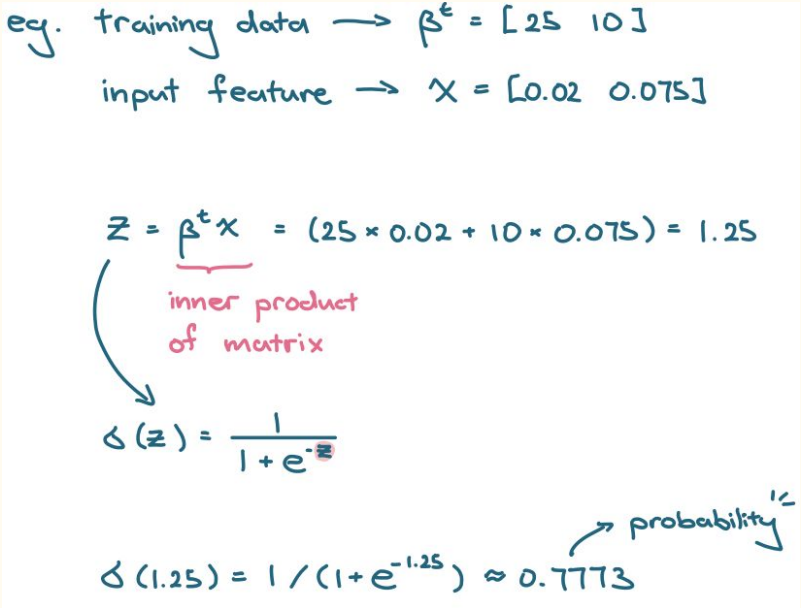
① $L \rightarrow 1$

② $k(t - t_0) \longrightarrow z$

Simple example on logistic regression

What are we regressing actually ? How exactly is logistic regression work ??

How do we convert the input features (discrete or continuous) to probability ???



key :

we treat z as a linear combination of input variables

beta : weighting

x : input feature

So how do we determine the beta / weighting ?

Rationale of MLE

So how do we estimate the weighting inside the linear combination of z ?

$$P = 1 / (1 + e^{-z})$$

$$z = \ln (p / 1-p)$$

$\ln (p/1-p)$ → z → weightings

logit / log(odds)

Maximum Likelihood Estimation(MLE)

- Me use MLE to determine the best weightings in z
- Maximum Likelihood : maximize the likelihood
- (optimal way to fit a distribution to data)
- Imagine we have some observaion on hand, and we want the find the model with the best value of hyperparameter to best-fit our observation / dataset

So... what is likelihood ? Is it related to probability ?

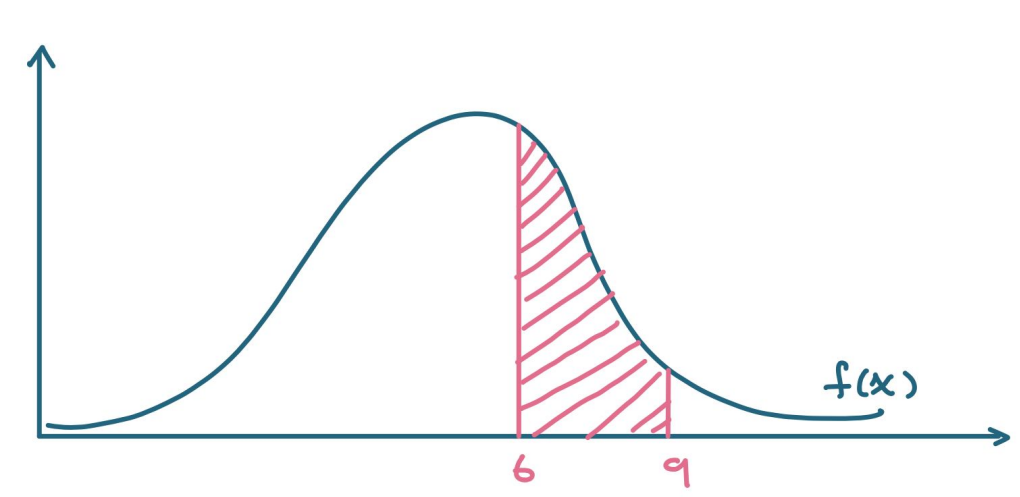
likelihood

Likelihood and probability share the same literal meaning, but slightly different statistical meaning (chicken and egg problem)

Probability : model parameters known → chance of outcome / occurence

Likelihood : observe sample → choose the model parameter by maximizing the probability of observing the sample in the model

Probability density function



$$P(6 \leq z \leq 9) = \int_6^9 f(x)\,dx$$

P(data | distribution)

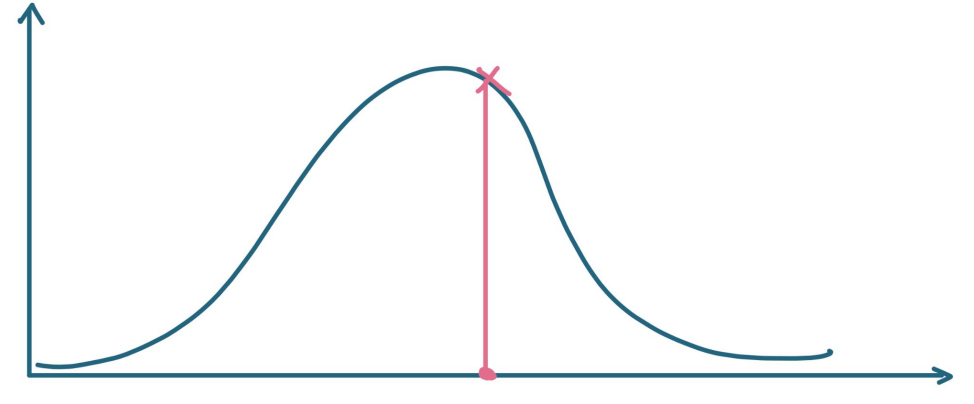
Probability :

Given a fixed distribution, the areas under curve (pdf) are probabilities.

Known distribution

Unknown sample

Probability density function



$L(\text{distribution} \mid \text{data})$

Likelihood :

Given a known sample, the y value of the point with distributions that can be moved is likelihood.

Known sample

Unknown distribution

Likelihood vs Probability (Example)

Example : Tossing a coin

Probability of getting a head $= 0.5$ (if it is fair)

→ assume P(heads) $= 0.5$

Imagine we flip the coin 1000 times, it only lands on head 77 times.

The **likelihood** that it is a fair coin is small.

→ obs : 77/1000 | likelihood of P(heads) $= 0.5$ is low

They are different points of view of the same scenario.

Likelihood vs Probability (Example)

Suppose a casino claims that the probability of winning money on a certain slot machine is 40% for each turn.

If we take one turn , the **probability** that we will win money is 0.40.

Now suppose we take 100 turns and we win 42 times. We would conclude that the **likelihood** that the probability of winning in 40% of turns seems to be fair.

When calculating the probability of winning on a given turn, we simply assume that P(winning) =0.40 on a given turn.

However, when calculating the likelihood we're trying to determine if the model parameter P(winning) = 0.40 is actually correctly specified.

In the example above, winning 42 times out of 100 makes us believe that a probability of winning 40% of the time seems reasonable.

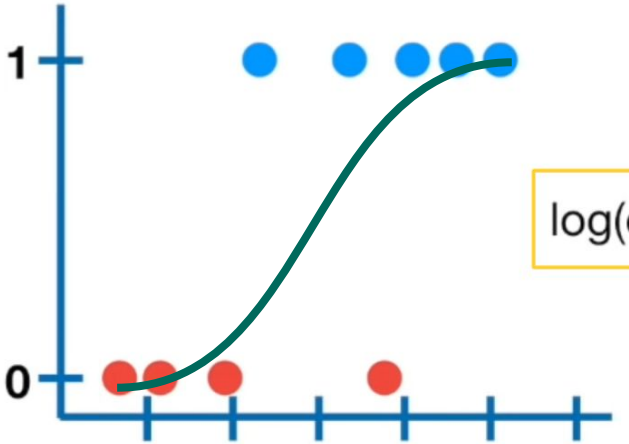
Maximum Likelihood Estimation(MLE)

So now we know what is likelihood, then the next question is :

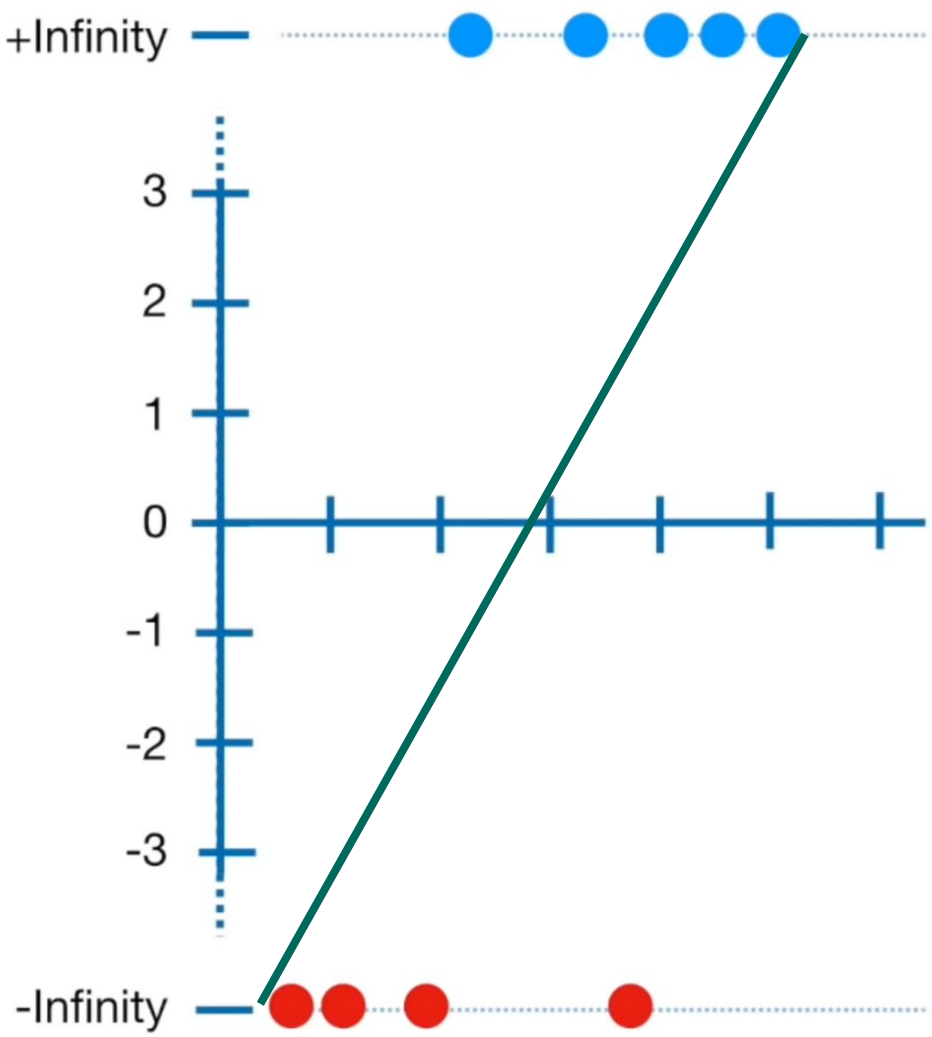
how do we do MLE for logistic regression ?

Flow :

(1) draw a random ‘candidate’ best-fit line to estimate
(2) transform probability axis to logits (or someone called log(odds))
(3) project all data to the corresponding or logits log(odds)
(4) calculate the likelihood of the model (cost function)
(5) using MLE to find the curve with maximum likelihood

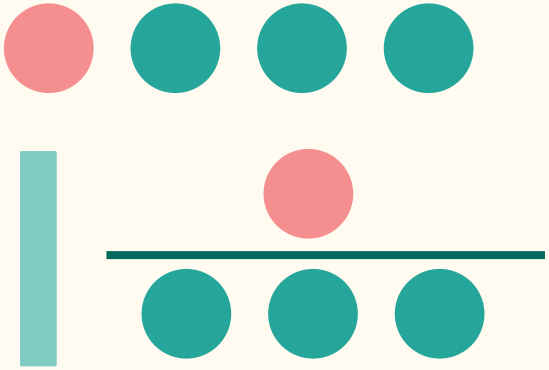


log(odds)



Why log(odds) ?

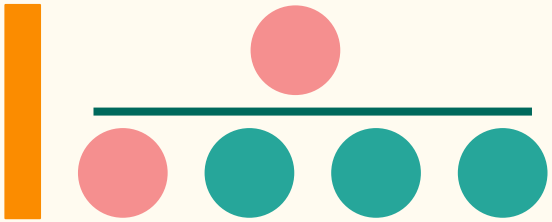
odds are not probability



ratio of something happening

not happening

odds



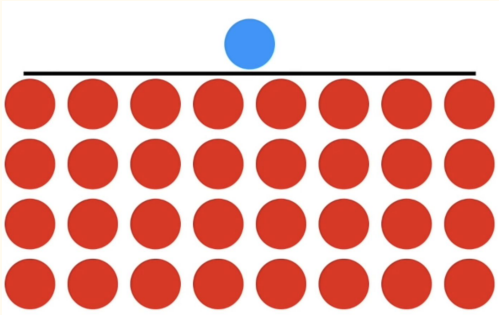
ratio of something happening

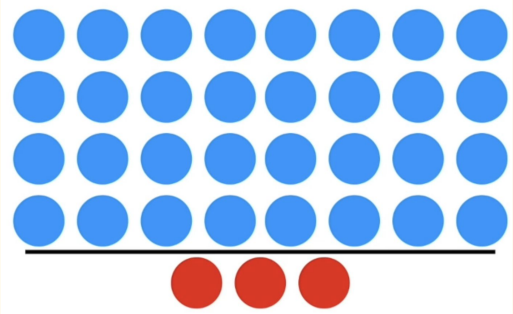
ALL outcomes

probability

Odds

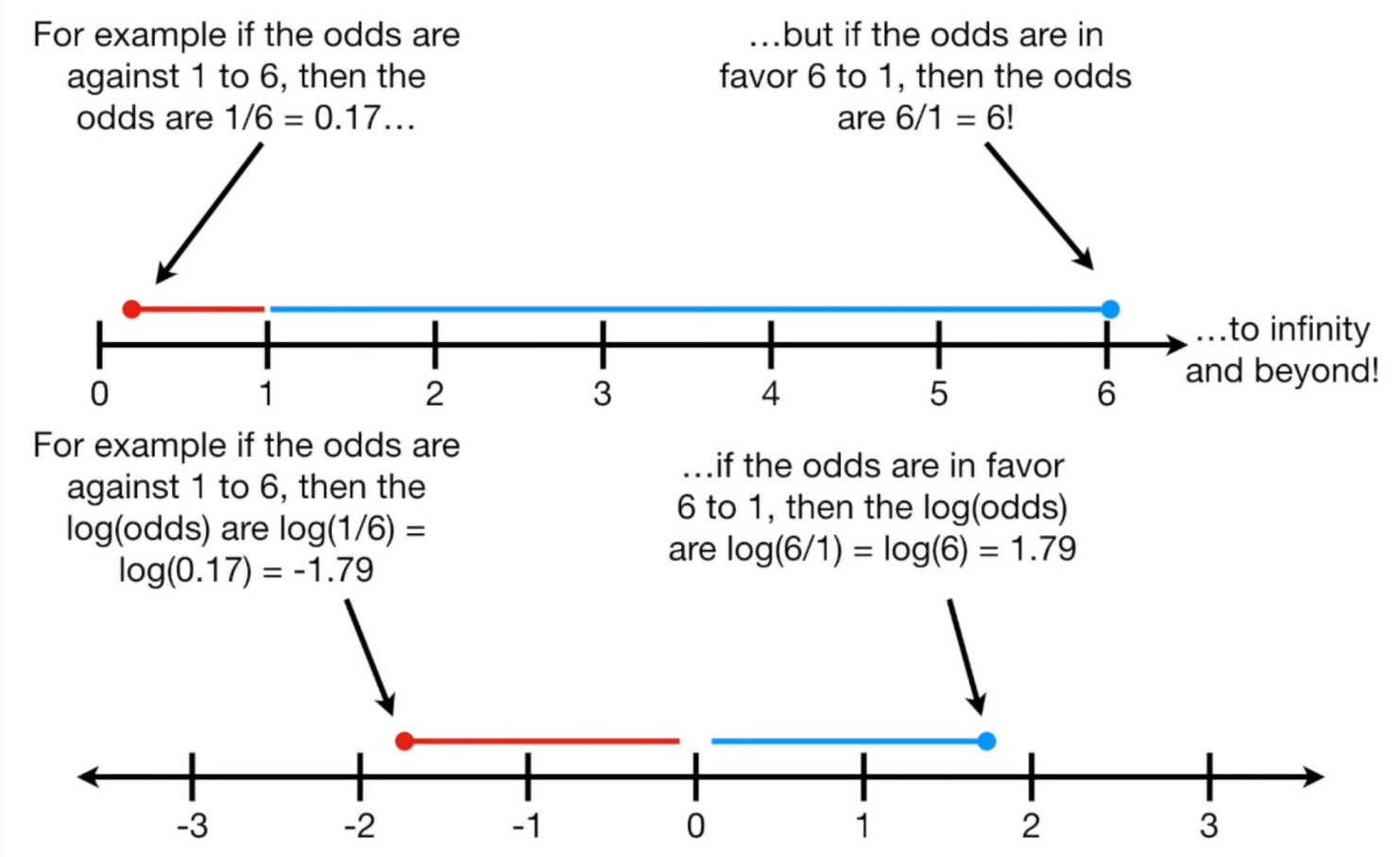
range of odds : (0, inf)

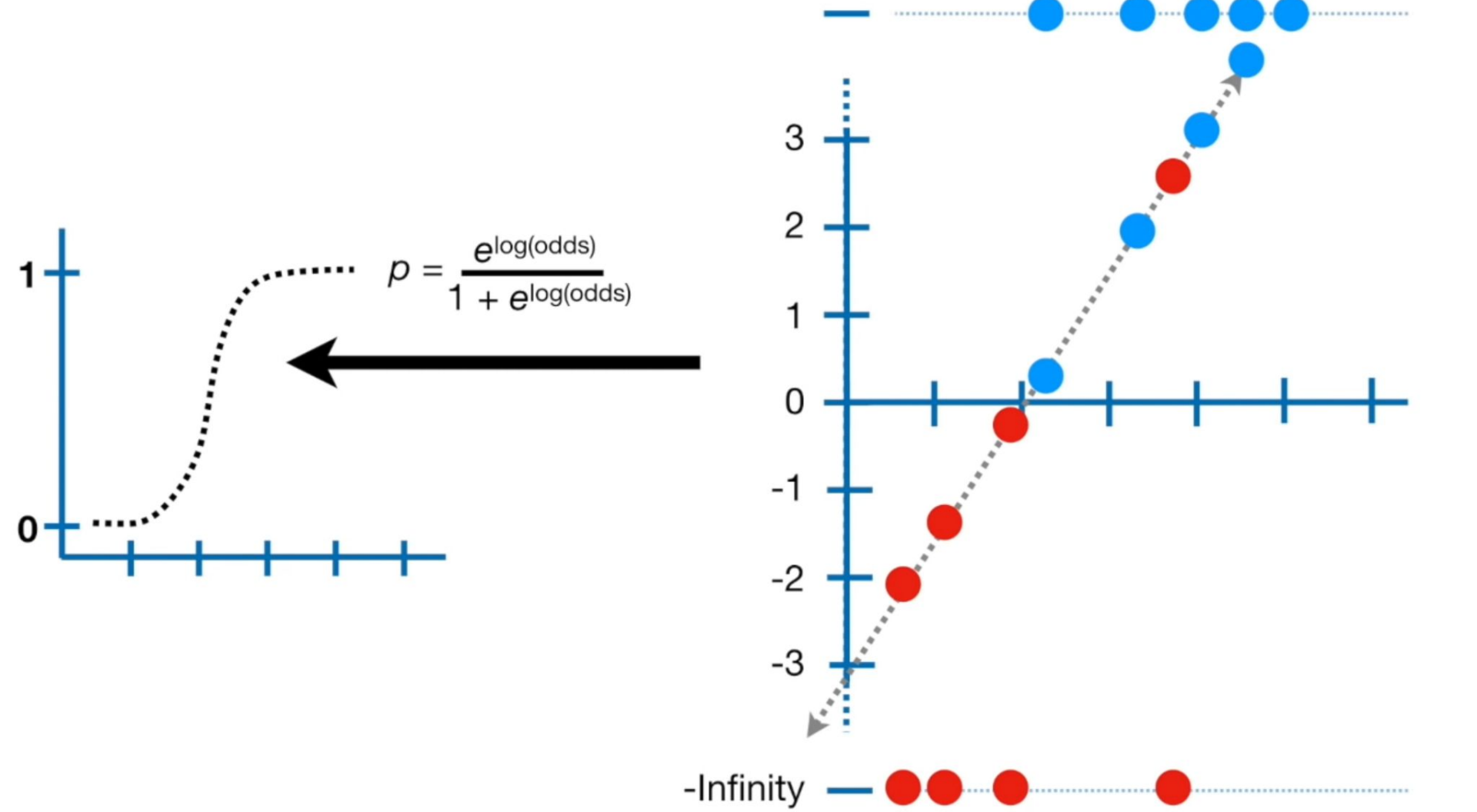




Asymmetry makes it difficult to compare the odds → take log

log (odds)


For example if the odds are against 1 to 6, then the odds are 1/6 = 0.17…
…but if the odds are in favor 6 to 1, then the odds are 6/1 = 6!
0
1
2
3
4
5
6
…to infinity and beyond!
For example if the odds are against 1 to 6, then the log(odds) are log(1/6) = log(0.17) = -1.79
…if the odds are in favor 6 to 1, then the log(odds) are log(6/1) = log(6) = 1.79
-3
-2
-1
0
1
2
3


1
0
$p = \frac{e^{\log(odds)}}{1 + e^{\log(odds)}}$
3
2
1
0
-1
-2
-3
-Infinity

Estimate p with log(odds)

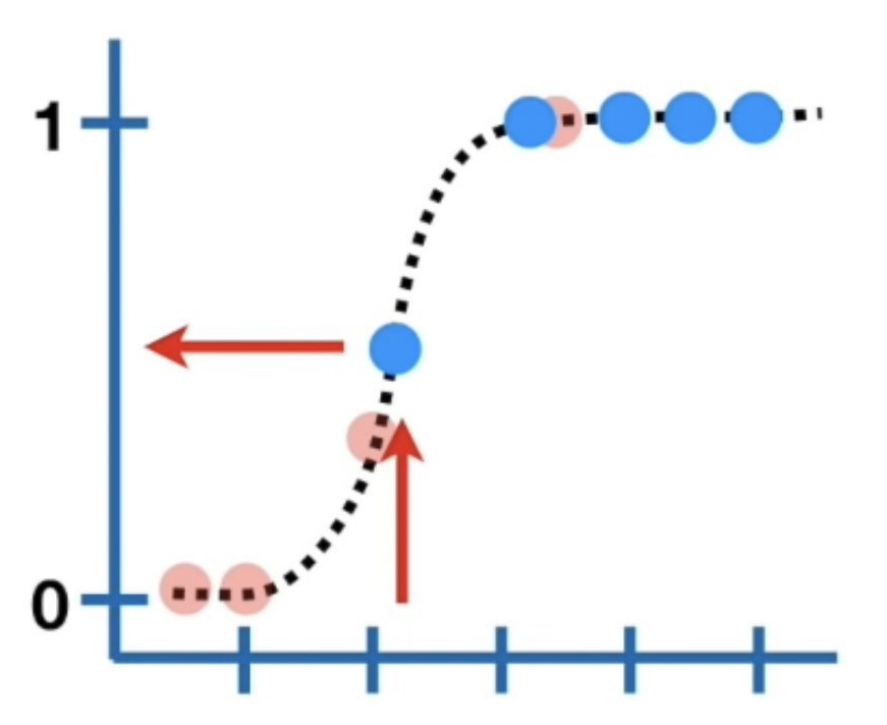
log(odds) → estimate p → weightings in z

$$\text{odds} = \frac{p}{1-p}$$

$$p = \text{odds} \,/\, 1 - \text{odds}$$

$$p = e^{\ln(\text{odds})} \,/\, 1 - \ln(\text{odds})$$

Estimate the likelihood



Calculate the likelihood of the curve :

y-value of the data point is the corresponding likelihood of the curve.

(Explanation : since probability is not calculated by the integral of area under curve, but instead it is just the specific value under a continuous probability density function, so we treat it as likelihood)

Maximum likelihood in logistic regression (visualization)

Extension : MLE in fitting other distributions

Logistic regression with Scikit-learn

Dependencies / Package
sklearn > linear_model > LogisticRegression
sklearn.metrics > confusion_matrix, accuracy_score
sklearn.feature_selection > f_regression

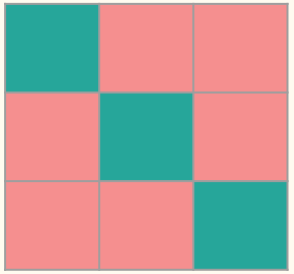
```
# Create linear regression object
regr = linear_model.LogisticRegression()

# Train the model using the training sets
regr.fit(X, y)

# Make predictions using the testing set
y_pred = regr.predict(X)
```

3 key steps

1. Create object (regression model)
2. Train model.fit(X, y)
3. Test and predict model.predict(X)

Example 4: wine dataset

Confusion matrix and accuracy quick recap



Class 0

Actual \ Predicted			
	TP	FN	FN
	FP	TN	TN
	FP	TN	TN

Class 1

Actual \ Predicted			
	TN	FP	TN
	FN	TP	FN
	TN	FP	TN

Class 2

Actual \ Predicted			
	TN	TN	FP
	TN	TN	FP
	FN	FN	TP

Confusion matrix and accuracy quick recap

Accuracy

$$\frac{T}{ALL} = \frac{TP + TN}{TP + FP + TN + FN}$$

F-value and P-value (statistics)

P-value : for checking the correlation between inut features and variables

(Take a level of significance(alpha = 0.05 / 0.1), when p-value is smaller than p-value, you are statistically confidence (1 - alpha) enough to tell that they are related)

f- value : step to calculate p-value when the test follows F distribution.

F-test is used to test whether two samples from different normal distributions have the same variance.

related concept : correlation coefficient, f-score, t-distribution,t-value , P-value

What is clustering ?

Imagine :

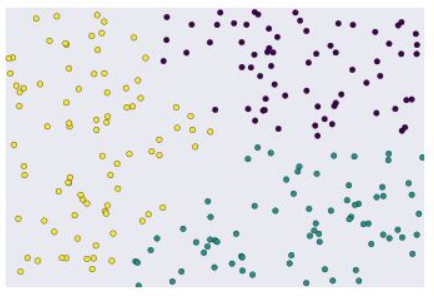
When you're trying to learn about something, say music, one approach might be to look for meaningful groups or collections. You might organize music by genre, while your friend might organize music by decade. How you choose to group items helps you to understand more about them as individual pieces of music.

We do the exact same thing in ML :

we **group things with similar properties** to understand the dataset.

Clustering : grouping unlabelled datasets.

When to use clustering ?

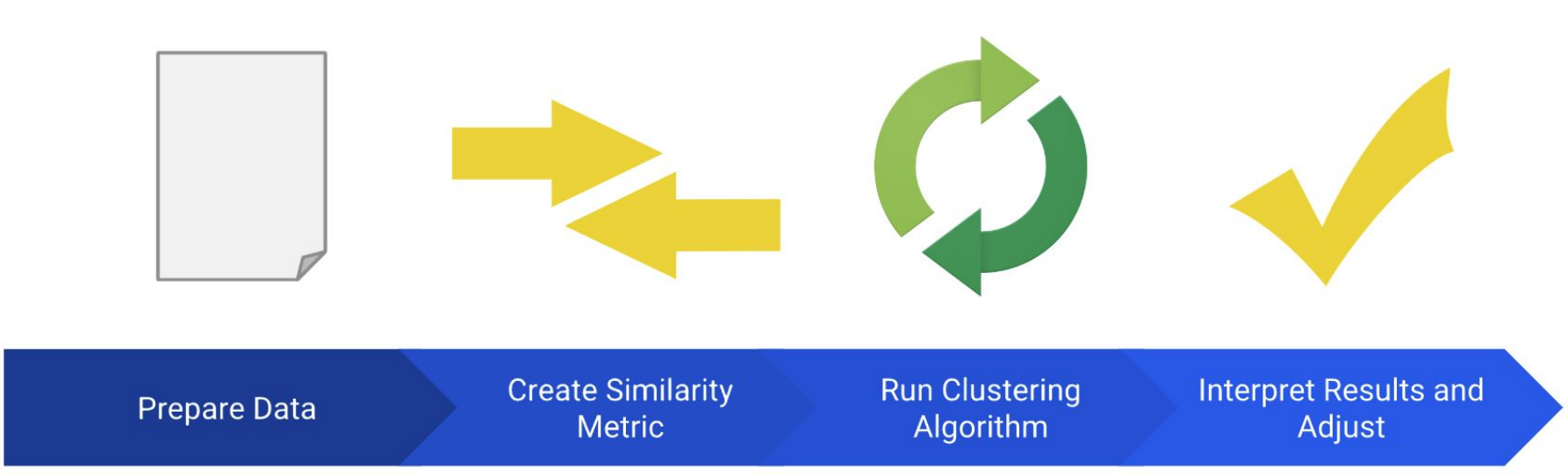


Unlabeled dataset → unsupervised learning : clustering

Labeled dataset → supervised learning : classification

- To group data, we have to figure out **similarity**.You can measure similarity between examples by combining the examples' feature data into a metric, called a **similarity measure**.
- When each example is defined by one or two features, it's easy to measure similarity. For example, you can find similar books by their authors.
- As the number of features increases, creating a similarity measure becomes more complex.

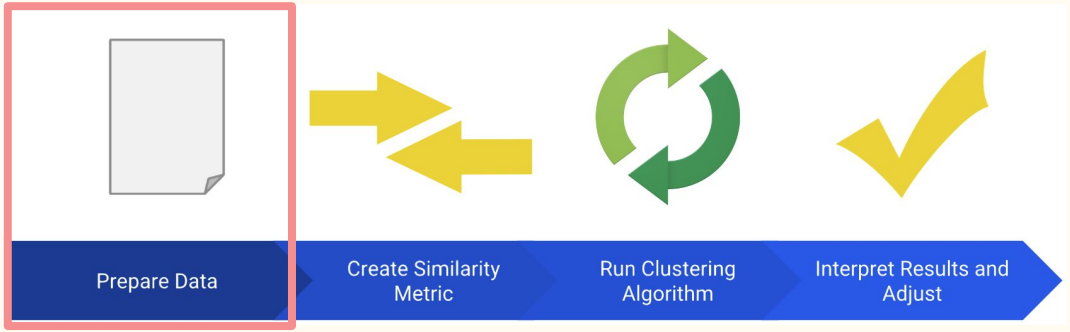
workflow of clustering



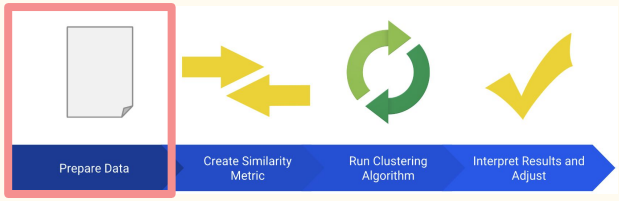
Workflow (1) : pre-processing

1. Normalization
2. Scaling (StandardScaler in sklearn)
3. Transformation of data (Eg. dimensionality reduction | PCA)

Key : ensure that the prepared data lets you accurately calculate the similarity between examples



Workflow (2) : similarity measure



Compute similarity for data :

For a simplified example, let's calculate similarity for two shoes with US sizes 8 and 11, and prices 120 and 150.

Action	Method
Scale the size.	Assume a maximum possible shoe size of 20. Divide 8 and 11 by the maximum size 20 to get 0.4 and 0.55.
Scale the price.	Divide 120 and 150 by the maximum price 150 to get 0.8 and 1.
Find the difference in size.	$0.55 - 0.4 = 0.15$
Find the difference in price.	$1 - 0.8 = 0.2$
Find the RMSE.	$\sqrt{\frac{0.2^2+0.15^2}{2}} = 0.17$

Workflow (2) : similarity measure

If the dataset involves categorical features :

- Single valued (univalent), such as a car's color ("white" or "blue" but never both)
- Multi-valued (multivalent), such as a movie's genre (can be "action" and "comedy" simultaneously, or just "action")

Similarity measure : Jaccard Index

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

Examples:

- [“comedy”,”action”] and [“comedy”,”action”] = 1
- [“comedy”,”action”] and [“action”] = ½
- [“comedy”,”action”] and [“action”, "drama"] = ⅓
- [“comedy”,”action”] and [“non-fiction”,”biographical”] = 0

Workflow (2) : similarity measure

In python, similarity measure is optimized under pandas dataframe methods :

1. Pearson
2. Spearman
3. Kendall tau

```
df.corr(methods = ‘pearson’/ ‘spearman’/ ‘kendall’)
```

Calculation

$$r = \frac{\sum_{i=1}^{n}(x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \overline{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \overline{y})^2}}$$

Pearson

$$\rho = \frac{\sum_{i=1}^{n}(a_i - \overline{a})(b_i - \overline{b})}{\sqrt{\sum_{i=1}^{n}(a_i - \overline{a})^2}\sqrt{\sum_{i=1}^{n}(b_i - \overline{b})^2}}$$

Spearman

$$\tau = \frac{(\text{number of concordant pairs}) - (\text{number of discordant pairs})}{(\text{number of pairs})} = 1 - \frac{2(\text{number of discordant pairs})}{\binom{n}{2}}$$

Kendall tau

Scale of measurement

Ratio scale : Area / Volume [Continuous]

Interval Scale : 9 / 10 oclock [Continuous]

Ordinal Scale : course Evaluation rank 1 - 5 [Categorical]

Nominal Scale : 0 - Female ; 1 - Male [Categorical]

Continuous vs Continuous : Pearson

Categorical vs Categorical : Spearman

Ordinal : Kendall tau

workflow (3) : clustering with Scikit-Learn

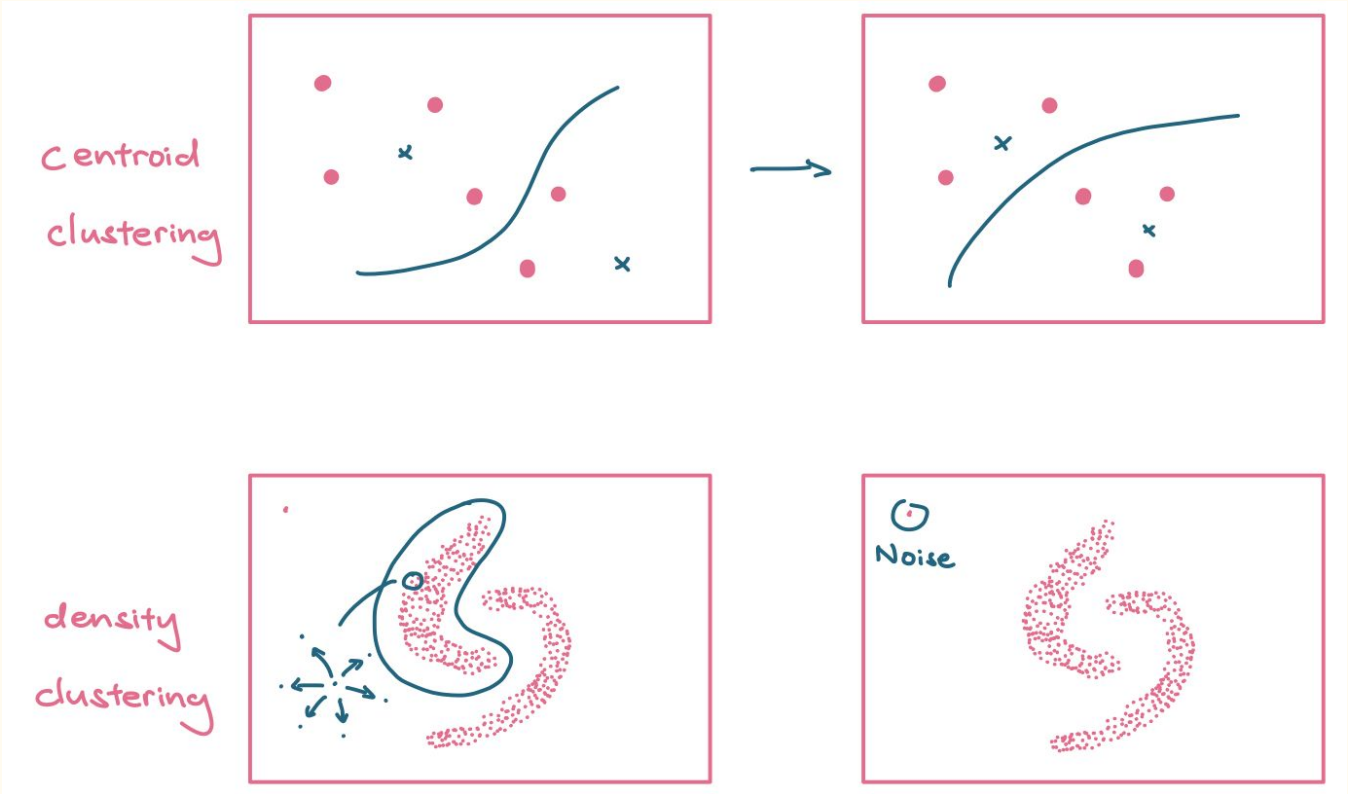
Clustering of unlabeled data can be performed with the module **sklearn.cluster**.

1. declare object
2. fit
3. predict and export labels

In this workshop, we will focus

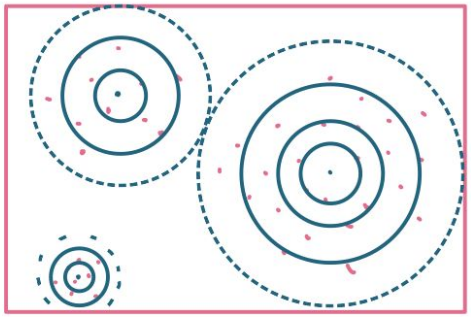
1. K-mean clustering
2. Mean-shift clustering
3. DBSCAN and HDBSCAN

4 basic types of clustering analysis

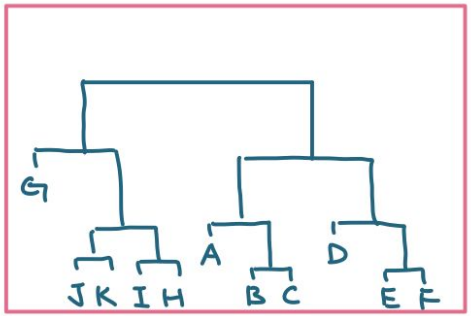


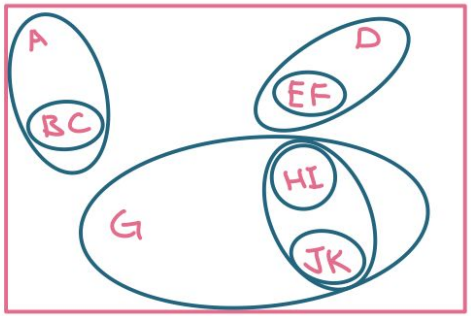
4 basic types of clustering analysis

distribution clustering

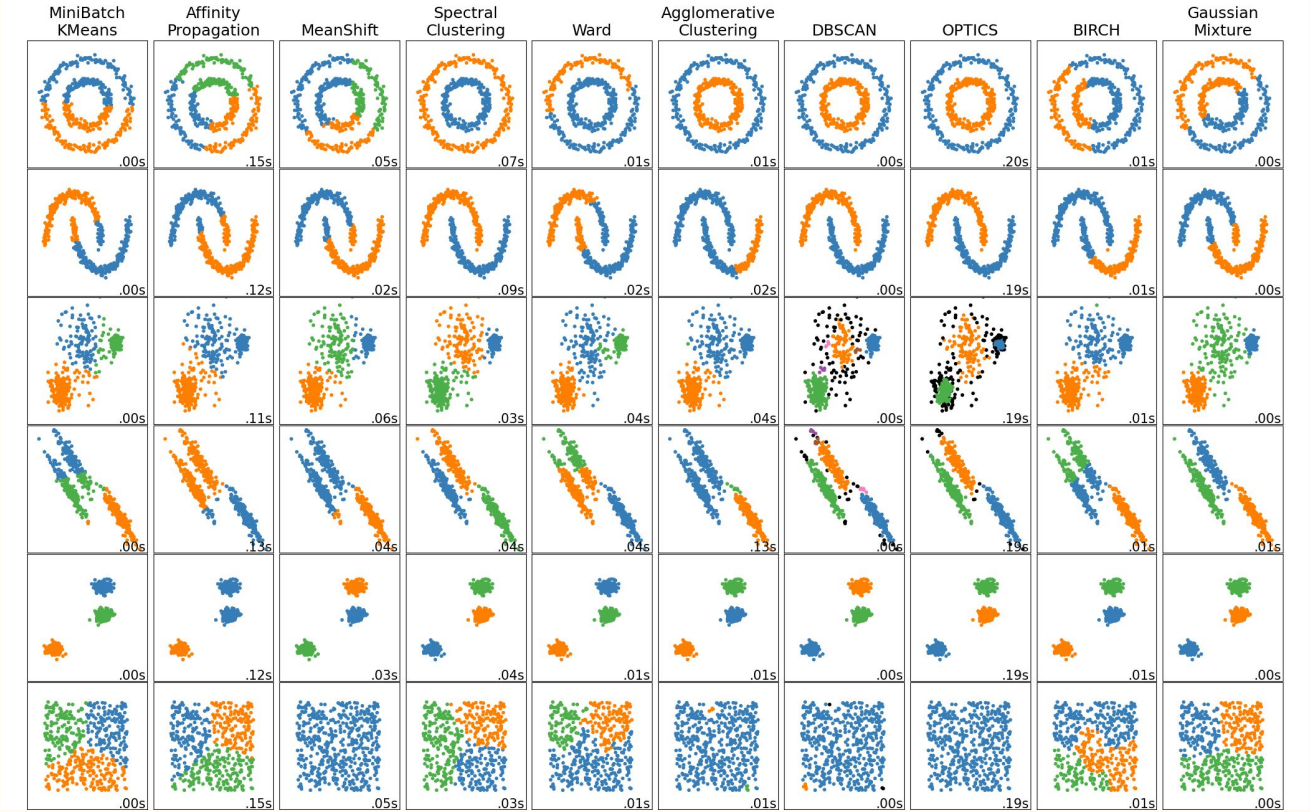


Connectivity clustering





Overview of clustering methods with Scikit-learn



K-mean clustering (concepts)

Methods :

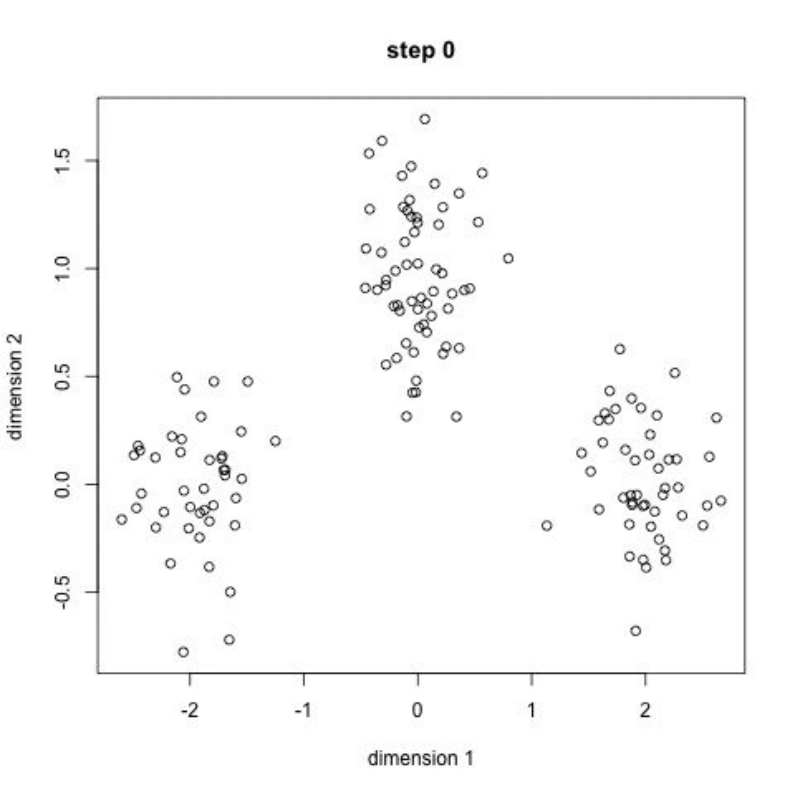
1. Grouping points to nearest centers
2. Define new centers (vector mean)
3. Repeat the iterations until convergence

Advantages :

Fast (Time complexity O(n))

Disadvantages :

1. Pre-defined number of clusters is required
2. Random choices of initial centers
   - Lack of consistency
   - Results may not be repeatable



K-mean clustering with sklearn

```
kmeans = KMeans(n_clusters=6)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
```

reminders:

k-mean requires pre-defined n_cluster

inertia and distortion

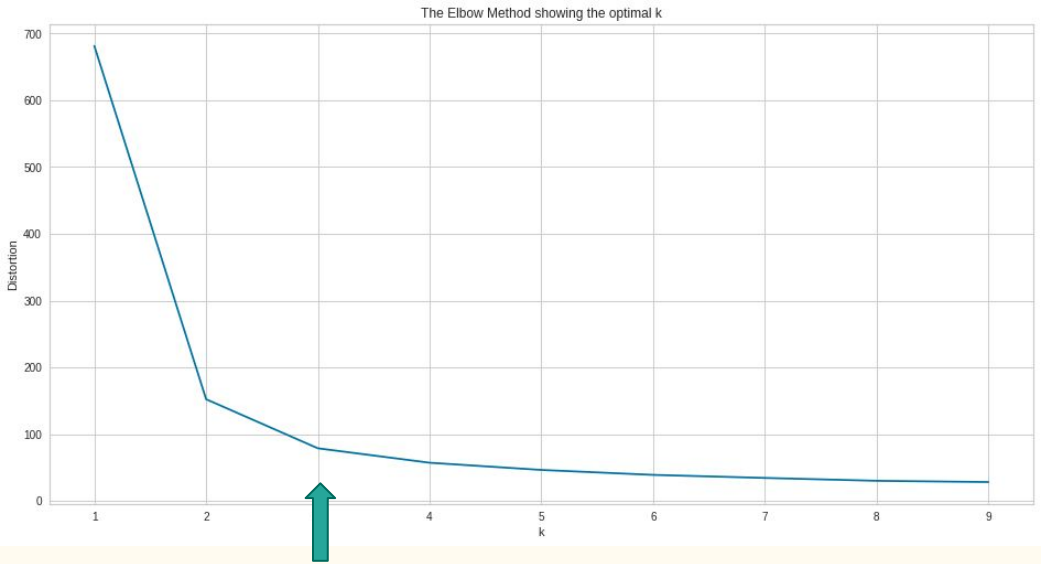
Inertia : sum of squared distances of samples to their closest cluster centre

Distortion : average euclidean distance from the centroid of the respective clusters

how to decide k ? - elbow method

minimise the inertia – adopt the number of cluster at elbow

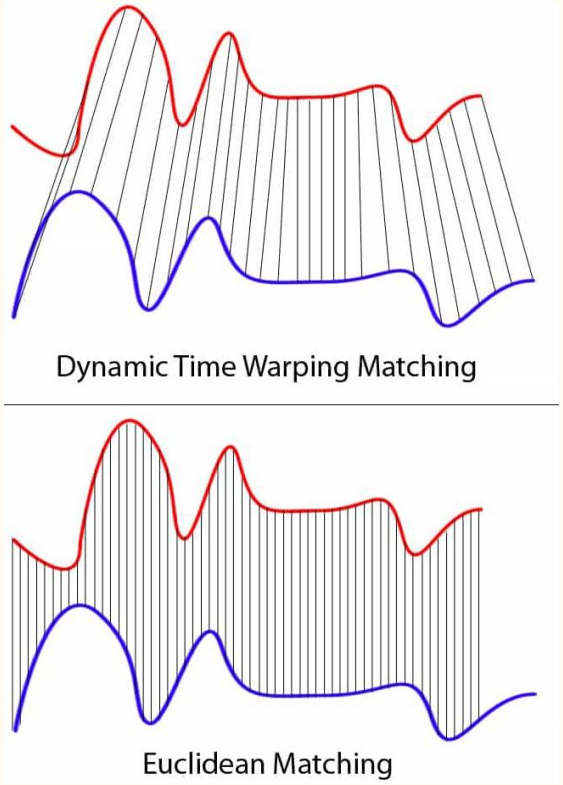
when the number of clusters starts increasing without sharp fall of inertia → elbow



Extension : k-mean clustering with time-series

When k-mean clustering used other metrics to determine the distance, it can be used in time-series data.

Eg. Dynamic time warping replacing euclidean distance to perform clustering between waveform



Evaluation : Silhouette Coefficient

Silhouette Coefficient or silhouette score is a metric used to calculate the goodness of a clustering technique. Its value ranges from -1 to 1.

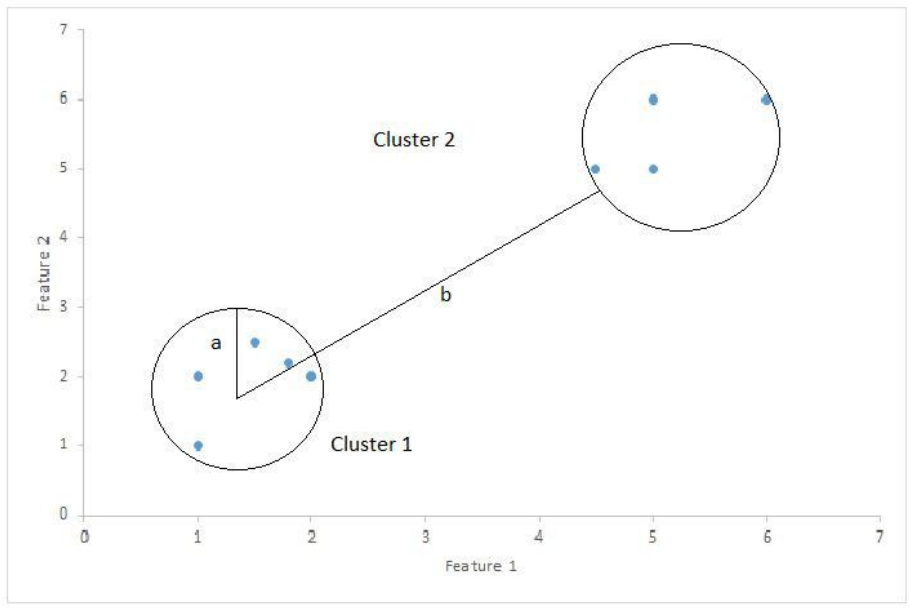
Silhouette Score = (b-a)/max(a,b)

a : average intra-cluster distance

(i.e. the average distance between each point within a cluster)

b: average inter-cluster distance

(i.e. the average distance between all clusters)



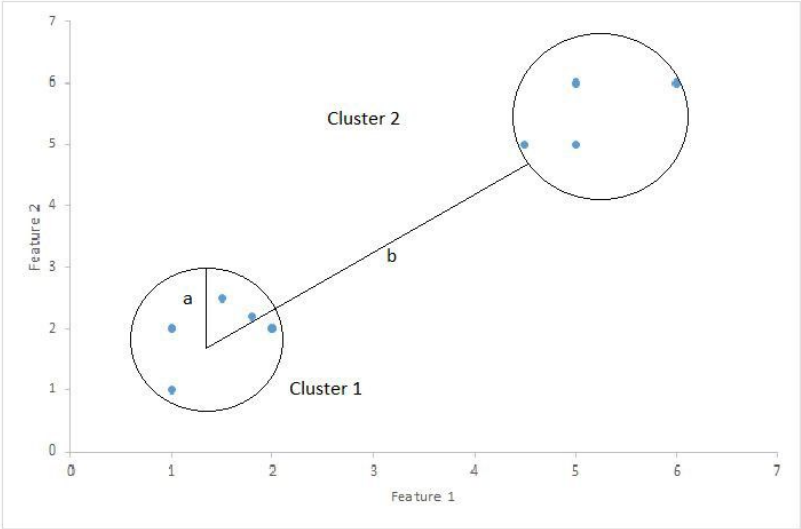
Evaluation : Silhouette Coefficient

1: Means clusters are well apart from each other and clearly distinguished.

0: Means clusters are indifferent, or we can say that the distance between clusters is not significant.

-1: Means clusters are assigned in the wrong way.

Silhouette Score = (b-a)/max(a,b)



Mean-shift clustering (concepts)

Methods:

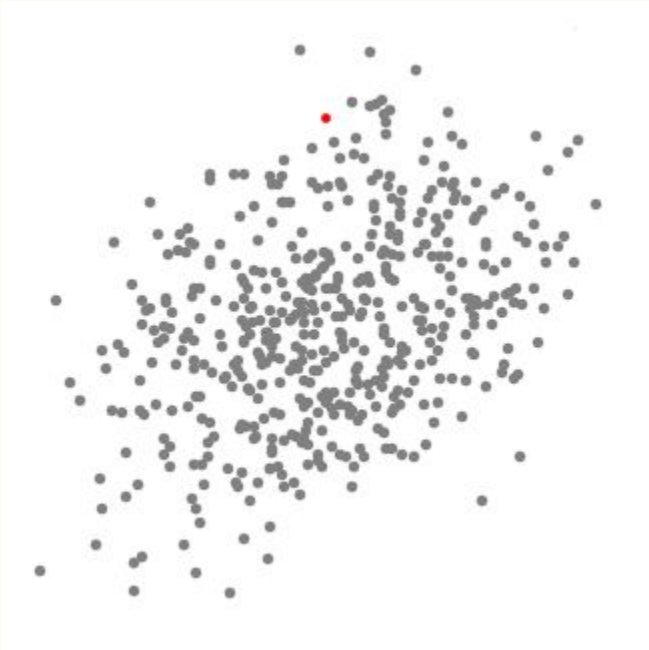
Shift the kernel to higher density regions iteratively until convergence (hill – climbing process)

Advantages:

1. No pre-defined number of clusters is needed
2. Intuitive to understand and fit
3. multi-processing

Disadvantages:

1. Selection of window size is non-trivial
2. All data are classified into cluster (even noise)



Process of mean-shift clustering

1. Create a sliding window/cluster for each data-point
2. Each of the sliding windows is **shifted towards higher density regions** by shifting their centroid (center of the sliding window) to the data-points' mean within the sliding window. This step will be r**epeated until no shift yields a higher density** (number of points in the sliding window)
3. Selection of sliding windows by deleting overlapping windows. When multiple sliding windows overlap, the window containing the most points is preserved, and the others are deleted.
4. Assigning the data points to the sliding window in which they reside.

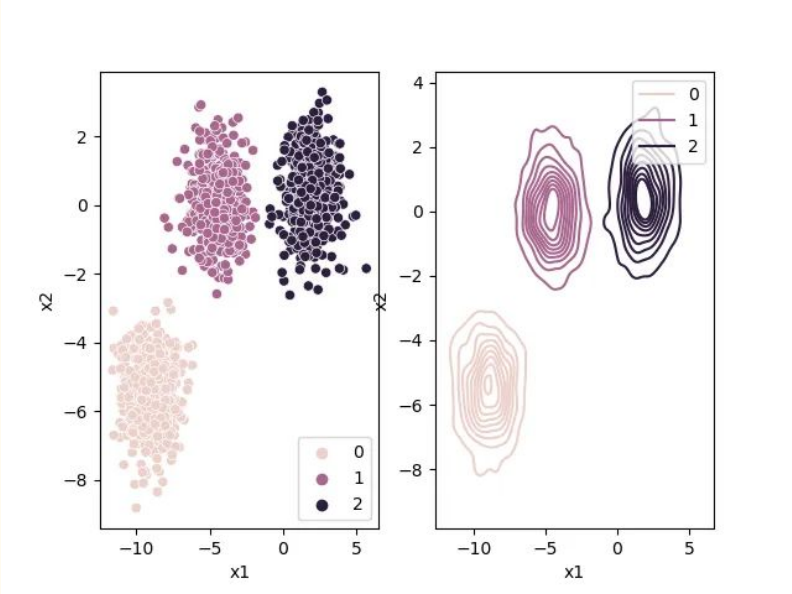
kernel density estimation

What Mean Shift is doing is :

shifting the windows to a higher density region

by shifting their centroid (center of the sliding window) to the mean of the data-points inside the sliding window.

We can also look at this by thinking of our data points as a probability density function.



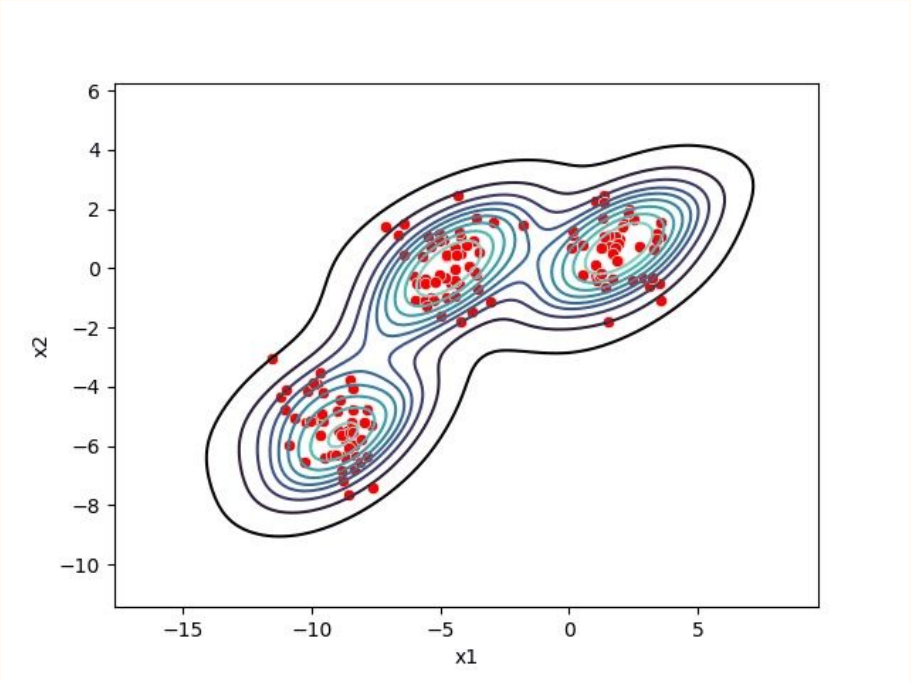
KDE (cont) and mean-shift clustering

Higher density regions correspond to regions with more data-points.

Lower density regions correspond to regions with fewer points.

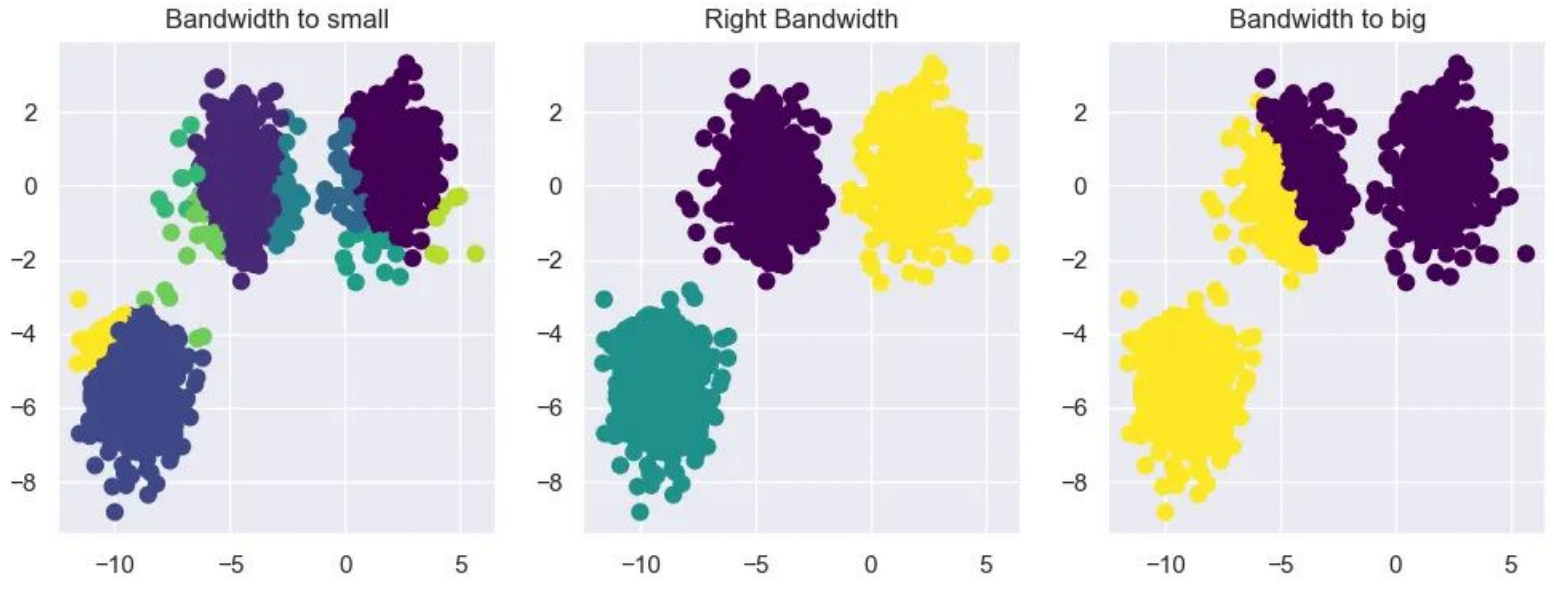
Mean Shift tries to find the high-density regions by continually shifting the sliding window closer to the peak of the region.

[gradient of pdf = 0 : reaching max density]



Bandwidth selection

Bandwidth : perception field or the size of sliding window of the ‘mean - shift’



Bandwidth estimate with sklearn

mean-shift clustering takes in the key argument of ‘bandwidth’ only

1. estimate bandwidth

   ```
   sklearn.cluster.estimate_bandwidth(X)
   ```

2. mean-shift clustering

   ```
   MeanShift(bandwidth=bandwidth)
   ```

Mean-shift clustering in image segmentation

We can apply mean-shift clustering on image to dissect the image by colours.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Methods:

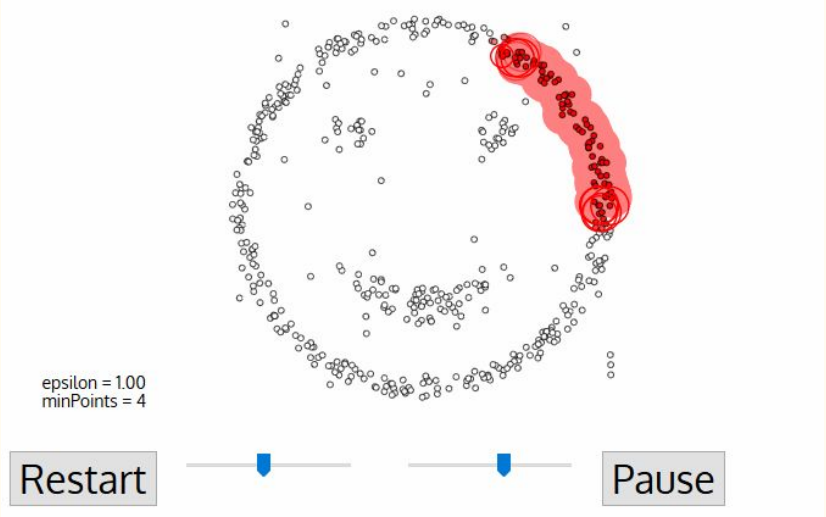
1. Select all neighboring points within pre-defined distances
2. Add the points into the cluster and repeat the iteration
3. Label all points either noise or cluster

Advantages:

1. No pre-defined number of clusters is needed
2. Identify outliers as noise

Disadvantages:

1. Performance reduce with data of varying density
2. High-dimension data (curse of dimensionality)



DBSCAN with sklearn

DBSCAN do not requires pre-defined number of clusters

Parameters::

eps : *float, default=0.5*

The maximum distance between two samples for one to be considered as in the neighborhood of the other. This is not a maximum bound on the distances of points within a cluster. This is the most important DBSCAN parameter to choose appropriately for your data set and distance function.

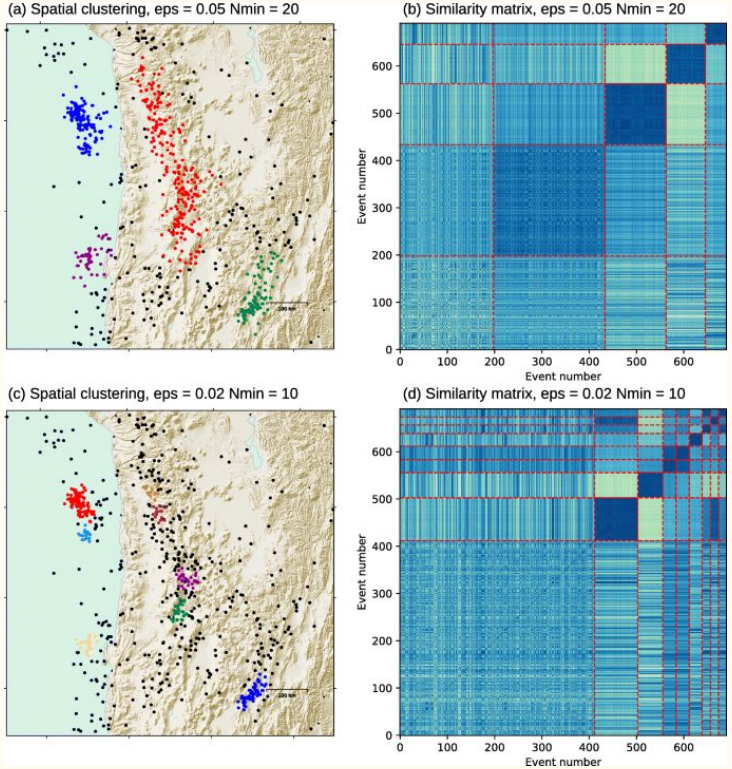
min_samples : *int, default=5*

The number of samples (or total weight) in a neighborhood for a point to be considered as a core point. This includes the point itself.

Actual application of DBSCAN on geophysics

There is application of DBSCAN on seismicity clustering

check it out here



Summary

K-mean :

requires pre-defined number of cluster → DBSCAN / mean-shift

perform bad in irregular shape → DBSCAN / mean-shift

can be used in high dimensional dataset

Mean-shift:

do not need pre-defined number of clusters

perform bad in un-even density → DBSCAN

DBSCAN :

Can classify noise